Fast algorithms for test optimization of core based 3D SoC[☆]Sabyasachee Banerjee^{a,*}, Subhashis Majumder^a, Debesh K. Das^b, Bhargab B. Bhattacharya^c^a Department of Computer Science & Engineering, Heritage Institute of Technology, Kolkata, 700107, India^b Department of Computer Science & Engineering, Jadavpur University, Jadavpur, Kolkata, WB 32, India^c Department of Computer Science & Engineering, Indian Institute of Technology Kharagpur, Kharagpur, WB 700108, India

ARTICLE INFO

Keywords:

SoC

TSV

TAM

Wrapper

ABSTRACT

The use of 3D-IC technology has become quite widespread in designing core-based systems-on-chip (SoCs). Concomitantly, testing of cores and inter-layer through-silicon-vias (TSVs) spanning through different layers of 3D chips has become an important problem in the manufacturing cycle. Testing 3D-SoCs is more challenging compared to their 2D counterparts because of the complexity of their design and power management issues. Also, the test procedure demands substantially more power than what is required in the normal functional mode, and hence, stringent thermal constraints during test need to be fulfilled to safeguard future performance and reliability of the chip. Since the overall 3D infrastructure depends on routing layer assignments, core allocation, and the geometry of TSV locations, these parameters should be given due consideration while designing the test-access-mechanism (TAM) that aims for minimizing overall test time satisfying power and TSV constraints. In this paper, we present a three-stage algorithm for reducing the test time in automated post-bond core-based 3D-SoCs, under a set of given constraints on test power, TAM-width, and the number of available TSVs. The proposed algorithm, when run on several ITC-02 SoC benchmarks, outperforms the algorithms presented in earlier work with respect to CPU-time, and additionally, reduces test time in many instances.

1. Introduction

A system-on-chip (SoC) is designed by assembling a number of pre-designed Intellectual-Property (IP) cores on different layers of a 3D chip and interconnecting them by using routing layers and through-silicon-vias (TSVs). These cores usually comprise various functional blocks such as processors, memories, buses, A/D-converters, DSP units, network controllers, peripherals, arbiters, or other application-specific hardware blocks. TSVs serve as vertical connectors that facilitate routing of signals, power/ground, thermal sinking, clock signals, and test data across different layers [1]. Today's process technologies now allow SoCs and embedded systems to integrate multiple cores as a single system [2]. As the complexity of the overall system continues to escalate, the need for testing becomes more pronounced. In a chip with 3D architecture, both test data volume and test time tend to increase manifold and the test procedure has to be performed via specially designed test-access-mechanism (TAM). In such a scenario, taking necessary steps to reduce test time as well as test power turns out to be an integral part of the SoC design process. In particular, test power consumption should not be allowed to exceed a certain

threshold to prevent possible thermal damage of the chip during test application. Therefore, the test sets for different cores [3] present in the SoC should be designed and scheduled in such a way that the given power constraint is not violated while attempting to minimize the overall test time.

In order to facilitate testing, an SoC is usually equipped with Test-access mechanism (TAM) and wrappers [4,5]. Test data is transmitted from their source to cores and from the cores to the test-response sink using TAMs [6]. Core wrappers serve as an interface between a core and TAM. In 3D SoCs, cores are allocated to different layers and a 3D-TAM is needed to transport test data to various layers from the pins and to the sinks, both placed at the bottommost layer. 3D-TAMs need to utilize the TSVs for vertical transport of test stimuli and test responses. However, TSVs increase fabrication cost, chip area, and often cause IR-drop at coupling points. Therefore, the test-access architecture should aim for using TSVs as few as possible.

The optimization of test time and test power involves obvious conflicts in a 3D SoC. For example, multiple cores with independent data links can be tested simultaneously to reduce test time. However,

[☆] A brief preliminary version of this paper was published in Banerjee et al. (2016).

* Corresponding author.

E-mail addresses: sabyasachi.banerjee@heritageit.edu (S. Banerjee), subhashis.majumder@heritageit.edu (S. Majumder), debeshd@hotmail.com (D.K. Das), bhargab@cse.iitkgp.ac.in (B.B. Bhattacharya).<https://doi.org/10.1016/j.vlsi.2020.11.009>

Received 17 February 2020; Received in revised form 3 November 2020; Accepted 10 November 2020

Available online 30 November 2020

0167-9260/© 2020 Elsevier B.V. All rights reserved.

concurrent test application to several cores lying in close geometric proximity may cause high thermal dissipation thus exceeding the allowable peak temperature. Also, test application in sparsely located cores over a long time-interval may violate the allowable average temperature. Hence, efficient test scheduling is needed for thermal management so that any permanent damage to the chip can be avoided. A thermally efficient core placement may overcome these problems during testing [7].

In this paper, we address the problem of minimizing the test time of an SoC by assigning suitable TAM-partitions to different cores. Those belonging to the same partition can be tested by adopting a strategy of serial and parallel scheduling. Additionally, we allocate the cores to layers in such a way that the thermal gradient of the corresponding TAM-partitions observed during test application follows a non-increasing pattern from bottom-to-top. As most of the heat is drained out from the bottommost layer of the chip, this means fewer thermal TSVs will be required for dissipation during testing, and it would be easier to fulfill the constraints on the number of available TSVs. Moreover, our algorithms are flexible; they can be tuned either to generate near-optimal TAM-partitions for a given TAM-width, or to work with a pre-determined TAM-partition as used by previous authors. The time required in pre-bond testing is, however, ignored in this paper.

The rest of this paper is organized as follows: Section 2 reviews prior work related to this area. A sample TAM-architecture is discussed in Section 3. The formulation of the problem, proposed algorithm and experimental results are presented in Section 4, Section 5, and in Section 6, respectively. Conclusions with possible future directions are reported in Section 7.

2. Prior work

One approach to optimize test time for 3D SoC was involved the use of the Integer Linear Program (ILP) and Mixed ILP (MILP) solvers [8–10]. However, it is difficult to use them for larger SoCs as in general they are computationally quite expensive. Comparatively greedy algorithms generally perform much better in terms of CPU time. Pradhan et al. [11] suggested a greedy heuristic to solve this problem, but they considered only two layers and also their approach did not consider the issues related to power management. In this work, we generate an initial partition by the Bin packing method as done by Iyengar et al. [12] and then we use a cost-based heuristic to optimize it. Banerjee et al. [13] suggested another greedy based heuristic approach along with placement to deal with power management with test time reduction. Wu et al. [10] and Girard et al. [14] had suggested power-constrained test scheduling, where they considered a threshold over the test power consumption of the SoC. There were a few earlier works, which were concentrated on scheduling test procedures [15–18]. A recent work by Sengupta et al. addresses a test planning technique under power constraints for stacked and non-stacked ICs [19]. They had considered the cores of an SoC for testing, for which scan chains are being used. They also had proposed a cost model for the test planning, but no details related to power consumption or thermal parameters were reported in their article. Cui et al. [20] presented an ACO based 3D bin packing model for 2D SoC. Our proposed algorithm not only requires less time for execution in comparison to some related earlier works [10,11,13], but also exhibits better results in most cases.

3. Motivational example

Consider the architecture shown in Fig. 1, where cores M_1 and M_2 are in one TAM-partition and placed in the bottommost layer (Layer 1), M_3 and M_4 in another TAM-partition and thereby placed in Layer 2 and finally the core M_5 is placed in the last TAM partition, which is placed in the topmost layer, (Layer 3). Let the total available TAM width for testing be W . Let T_1 be the total number of wires required under the

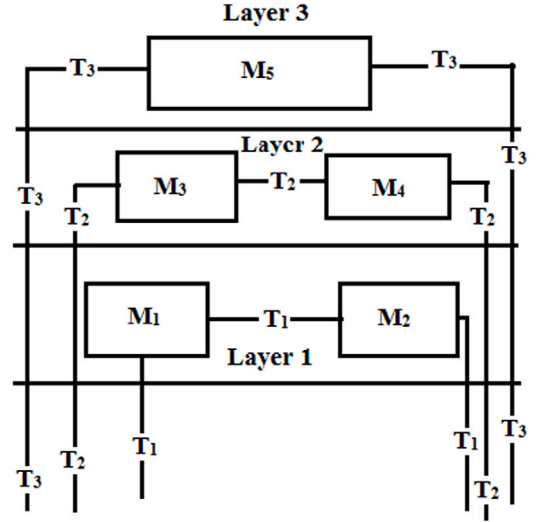


Fig. 1. Example of TAM usage with constraints in TSV.

TAM-partition (TAM_1) consisting of M_1 and M_2 , similarly T_2 be the total number of wires required for TAM-partition (TAM_2) consisting of M_3 and M_4 and finally T_3 be the number of wires required for the last TAM-partition (TAM_3) with only core M_5 (see Fig. 1). Since, TAM_1 is the bottommost layer, the cores at TAM_2 and TAM_3 we require 3D-TAMs called as test elevators [21]. For this, between Layer 1 and Layer 2 $2(T_2 + T_3)$ TSVs will be required - $(T_2 + T_3)$ TSVs will be required to bring up the test signals from Layer 1 to Layer 2 and rest to take them back to the sink, which is at Layer 1. Similarly, $2T_3$ number of TSVs will be required between Layer 2 and Layer 3.

Hence, for testing purpose number of TSVs required in the SoC is, $2T_2 + 4T_3$. Thus the TSV constraint (TSV_{max}) must be $\geq 2T_2 + 4T_3$. As $W = T_1 + T_2 + T_3$, the total number of test pins is $2(T_1 + T_2 + T_3)$. Now the TAM-partitions are used in such a way that in the bottommost layer the TAM partition having the highest width is placed. The rest of the TAM partitions are placed in other layers in the non-increasing order of their width that ensures the less requirement of the total number of TSVs. For high power-consuming cores, we try to place the TAM partitions towards the bottom, which will ensure a better heat sink. So, the bottommost TAM partition not only has the highest width, but also consumes maximum power. Finally, we have to ensure the minimum testing time by using a mix of serial and parallel test scheduling without violating the TSV-constraint.

4. Problem formulation

The problem of partitioning the cores of SoC into its different layers is mapped into a graph partitioning problem. An SoC is represented as a graph $G = (V, E)$, where $V = \{V_1, V_2, \dots, V_n\}$ be the set of vertices corresponding to the cores and E be the edges corresponding to the TAM-wires.

1. The set V is to be partitioned in such a way that

$$\bigcup_{j=1}^K TAM_j = V \text{ and}$$

$$\forall 1 \leq i, j \leq K, i \neq j, TAM_i \cap TAM_j = \emptyset,$$

where K be the total number of layers and TAM_j is the set of cores in the TAM-partition j . Let $Time_{TAM_j}$ be the total test time needed to test the TAM-partition j . Then to have minimum time for testing the SoC our objective is to minimize $\max_j (Time_{TAM_j})$.

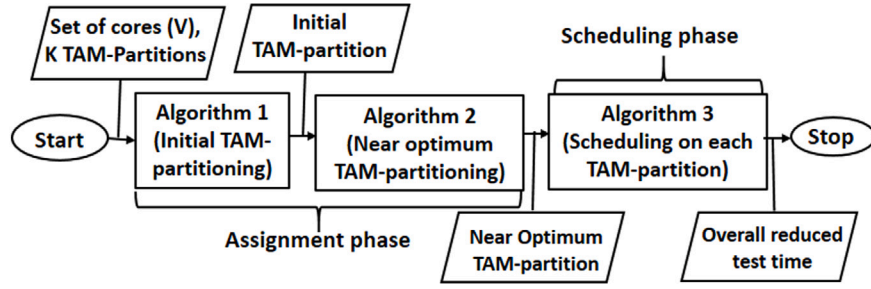


Fig. 2. Flow of our proposed technique.

2. Let $Pwr(V_i)$ be the power required during testing of the particular core V_i . Assuming the arbitrary partitioning of the cores, say a core V_i be placed in Tam_j . Let us define a cost function-

$$Cost(V_i, Tam_j) = \alpha * \max_j(Time_{Tam_j}) + (1 - \alpha) * Layer(Tam_j) * Pwr(V_i) \quad (1)$$

where $Layer(Tam_j)$ is the layer where Tam_j is placed. We have tried to place a single TAM-partition in a single layer in such a way that the TAM-partition with higher TAM width gets placed to closer to the bottommost layer. Otherwise, the number of TSVs is going to increase, which might affect the cost and reliability of the SoC. Note that a convex combination of two parameters *test time* and *power required* by the core is used to define the *Cost* function that we specify above. The contributions of these two variables are controlled by the parameter $\alpha \in [0, 1]$. When $\alpha = 1$ (0), optimization is done entirely on the basis of test time (power consumption). Moreover, note that in the *cost* function, the value of the power consumption is scaled by the layer number, so that the upper layers automatically get a lesser share of power. The rationale behind this is that the higher a layer is placed, the more difficult it is to dissipate the heat generated there. Let $Gain(V_i, Tam_j)$ be the gain in *Cost* by moving a core V_i from Tam_{prev} to Tam_j , so

$$Gain(V_i, Tam_j) = Cost(V_i, Tam_{prev}) - Cost(V_i, Tam_j) \quad (2)$$

We calculate a separate gain for each i, j , i.e., for each of the cores under different Tam_j . Finally, we define the global gain of each pass as

$$G = \max_{i,j}(Gain(V_i, Tam_j)) \quad (3)$$

In each step we move only that module (say V_{max}), which will give the maximum positive gain; ties will be broken arbitrarily by choosing one of the cores with the maximum positive gain. After moving the core V_{max} , we will again evaluate $Cost(V_i, Tam_j)$ for each of the cores and repeat the whole step again till no core can be found providing positive gain ($Gain(G)$). The same core is allowed to change partition more than once and even to come back to a partition it left in an earlier step. The method thus converges, as at every step we proceed only if there is a positive gain.

5. Proposed algorithm

We propose here a three phase technique, where the first two phases (Algorithms 1, 2) are used to generate the assignment of cores in different TAM-partitions (*Assignment phase*), whereas in the last phase, Algorithm 3 (*Scheduling Phase*) is applied on each of the TAM-partitions to minimize the overall test time. The overall flow of the algorithms can be found in Fig. 2.

If the TAM partitions of higher width are placed towards the upper layer, it will demand a higher number of TSVs for building the test

elevator. Thus the TAM partitions are first ordered in the order of non-increasing width to keep the TAM of higher width in the lower layer. Moreover, as the heat sinks are at the bottom, we try to assign power-hungry cores at the lower layer. The proposed algorithms for *Assignment phase* try to assign the cores among various TAM partitions in such a way that the overall test time of SoC is minimized compared to some earlier works [10,11]. Note that our algorithm has a very negligible run time also. The proposed method has two parts. First, Algorithm 1 performs an initial TAM-partitioning based on the 2D rectangular bin packing technique [12], with Width W and length T , where W and T correspond to TAM width and test time respectively. Our objective is to place the bins in a manner such that it would acquire as less area as possible. In the second part, Algorithm 2 uses the result of Algorithm 1. The *Gain* for each core is calculated, by placing it in each TAM-partition. The core with the maximum gain is moved from its original TAM-partition to another TAM-partition and the process is repeated until no positive gain be possible.

Algorithm 1: Initial Multi-Partitioning in 3D-SoC

input : A set of cores V , each core V_i with a set of K test times under K TAM-partitions ($Time(V_i, Tam_1), \dots$, and $Time(V_i, Tam_K)$), the total TAM width (W) and width of K TAM partitions ($W_1, \dots$, and W_K)

output: Assignment of the cores under a certain TAM partition

```

1 for each core  $V_p$  do
2   minimum  $\leftarrow W_1 * Time(V_p, Tam_1)$ ;
3    $P_p \leftarrow 1$ ; /* core  $V_p$  is placed in TAM partition  $P_p$  */
4   for  $q \leftarrow 2$  to  $K$  do
5     if (minimum > ( $W_q * Time(V_p, Tam_q)$ )) then
6       minimum  $\leftarrow W_q * Time(V_p, Tam_q)$ ;
7        $P_p \leftarrow q$ ;
8   end
9 end
10 end
  
```

Now the output of the Algorithm 2 is supplied to the Algorithm 3. Under the available TAM width W_j of the TAM partition Tam_i the cores are sorted in non-increasing order of their test times. Ties are resolved randomly. Based on the order we start to schedule cores in the TAM partition. For the first core V_f , we assign it as much TAM width $w (\leq W_j)$ as possible say, so that its test time may be minimized as much as possible to value say tt_f . However, TAM width vs. test time plot of the core V_f we consider the pareto optimal points. So that w corresponds to the minimum TAM width required to guarantee a test time of tt_f . If the full available TAM width (W_j) is not required by the core V_f , i.e., $w < W_j$, we chose the next core V_n in the sorted order and schedule it, in the same timestamp, if it can be tested using the remaining available TAM width ($W_j - w$), and in time $\leq tt_f$.

Furthermore, our attempt is to reduce the TAM width assigned to V_n as much as possible, which may increase the test time of it (tt_n), but not over the limit of tt_f . Now after the testing of V_n , we still may have

Algorithm 2: Convex Multi-Partitioning in 3D-SoC

input : TAM width (W), An SoC having a set of cores V , with each core V_i having a set of test times $Time(V_i, Tam_j)$, $1 \leq j \leq K$, the TAM-partition Tam_i where core V_i is initially assigned, width of K TAM partitions ($W_1, \dots$, and W_K , where $\sum_{i=1}^K W_i = W$)

output: Near optimum assignment of the cores ($V_i, V_i \in V$) to a certain TAM partition (Tam_j) maintaining TSV and thermal constraint

- Sort TAM partitions in non-increasing order according to there TAM-width;
- If TAM-widths are same then sort them in non-increasing order according to the power consumption of cores;
- for** $q \leftarrow 1$ to K **do**
- $Layer(Tam_q) \leftarrow q$;
- end**
- $maximum \leftarrow 0$;
- for each core** V_p **do**
- for** $q \leftarrow 1$ to K **do**
- evaluate $Cost(V_p, Tam_q)$;
- if** ($P_p = q$) **then**
- $Cost(V_p, Tam_{prev}) \leftarrow Cost(V_p, Tam_q)$;
- end**
- end**
- for** $q \leftarrow 1$ to K **do**
- evaluate $Gain(V_p, Tam_q)$;
- if** ($maximum < Gain(V_p, Tam_q)$) **then**
- $maximum \leftarrow Gain(V_p, Tam_q)$;
- $CoreToMove \leftarrow p$;
- $Tam_{Move} \leftarrow q$;
- end**
- end**
- end**
- if** ($maximum > 0$) **then**
- Place module $V_{CoreToMove}$ to Tam_{Move} ;
- update power dissipation, test time of TAMs;
- go to step 6;
- end**
- STOP**;

Table 1
TAM-width and test time of d695.

TAM width 3					
Core	1	2	3	4	5
Test time	155	5146	2507	11 129	64 070
Core	6	7	8	9	10
Test time	62 248	21 959	8721	8802	40 137
TAM width 21					
Core	1	2	3	4	5
Test time	38	809	2507	5829	9989
Core	6	7	8	9	10
Test time	9869	3359	4605	1416	7106

some time left $tt_f - tt_n$ (RemainingWindow in Algorithm 3) under the same remaining TAM width $W_j - w$ and same timestamp, which we can utilize by testing more cores serially with V_n and parallel with V_f .

Basically the Algorithm 3 is a modification of one of our earlier works [22]. For example, in Fig. 3, we try to show how the individual cores of d695 under total TAM width of 24 got scheduled for two different TAM partitions of width 21 and 3. After the execution of Algorithm 2 on d695, 6 cores (1, 2, 7, 10, 6 and 5) got scheduled serially

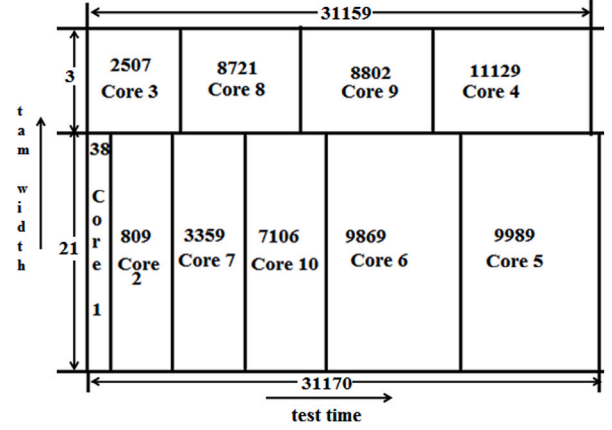


Fig. 3. Example of test scheduling with out Algorithm 3.

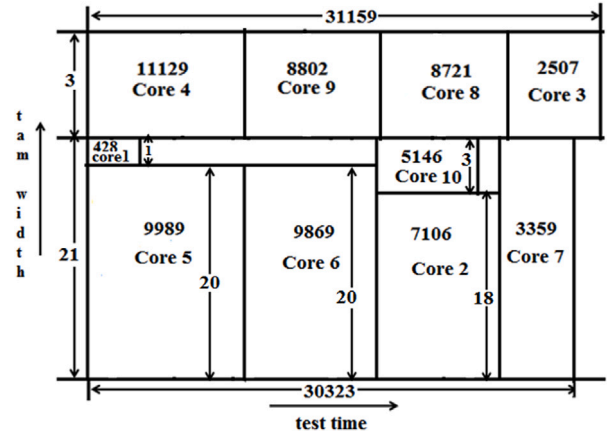


Fig. 4. Example of test scheduling with Algorithm 3.

Table 2
Power dissipation in ten cores of d695.

Core	1	2	3	4	5
Power	7.21	15.81	11.75	61.56	253.25
Core	6	7	8	9	10
Power	321.74	112.66	51.05	50.68	72.62

in the TAM partition, having TAM width 21 and rest 4 cores (3, 8, 9 and 4) got scheduled serially in the TAM partition, having TAM width 3.

So without implementing the Algorithm 3 the test time we get for d695 is $\max((2507 + 8721 + 8802 + 11129), (38 + 809 + 3359 + 7106 + 9869 + 9989)) = \max(31159, 31170) = 31170$ for total $3 \times 2 = 6$ number of TSVs. Now if we apply Algorithm 3 on this result we obtain a different scheduling explained in Fig. 4. The test times of core 1 and core 10 are stretched to be fitted in the same timestamp of core 20 and core 2 respectively. The test time we get here is $\max((2507 + 8721 + 8802 + 11129), (9889 + 9869 + 7106 + 3359)) = \max(31159, 30323) = 31159$ for same number ($3 \times 2 = 6$) of TSVs.

5.1. A typical example

We demonstrate here with an example of how different algorithms produce the desired output, in a step-by-step manner. Let us consider Table 1, which gives the test times for each of the cores when placed in the two given TAM-partitions having TAM widths of 3 and 21, respectively. Next consider Table 2 that gives the amount of power the

Algorithm 3: Scheduling cores in different TAM partitions

input : Cores (V_i) with their assignment to a certain TAM partition (Tam_j),
width of K TAM partitions ($W_1, \dots$, and W_K)

output: Total time needed to test the SoC ($TotTime$),
TAM width used for each core ($Width_{V_i}$),
Timestamp (t_{V_i}) in which core (V_i) of TAM partition (Tam_j) is tested.

```

1   $TotTime \leftarrow 0$ ;
2  for  $j \leftarrow 1$  To  $K$  do
3       $i \leftarrow 0$ ;
4       $V_{Tam_j} \leftarrow \emptyset$ ;
5      for each core  $V_i \in Tam_j$  do
6           $i \leftarrow i + 1$ ;
7           $V_{Tam_j} \leftarrow V_{Tam_j} \cup V_i$ ;
8           $isTest_{V_i} \leftarrow 0$ ;                                /* 1  $\Rightarrow$  core  $i$  is tested */
9      end
10      $Sort(V_{Tam_j} [ ])$ ;    /* Cores of TAM partition  $Tam_j$  sorted in descending order of test times for TAM width  $W_j$  */
11      $CurrentTstamp \leftarrow 0$ ;  $TotTime_j \leftarrow 0$ ;
12     while (All cores on TAM partition  $Tam_j$  are not yet tested) do
13          $w = 0$ ;  $W_{available} \leftarrow W_j$ ;  $CurrentTstamp \leftarrow CurrentTstamp + 1$ ;
14         for  $i \leftarrow 1$  To  $n_j$  do
15             /* consider cores in order */
16             if ( $isTest_{V_i} = 0$ ) then
17                  $w \leftarrow W_{available}$ ;
18                 if ( $w = W_j$ ) then
19                     /* 1st core in timestamp */
20                      $TstampSpan \leftarrow Time_w V_i$ ;  $TotTime_j \leftarrow TotTime_j + TstampSpan$ ;
21                 end
22                 else
23                     if ( $Time_w V_i > TstampSpan$ ) then
24                          $\text{continue}$ ;                                /* width left not enough, try next core */
25                     end
26                     while ( $Time_{w-1} V_i \leq TstampSpan$ ) do
27                          $w \leftarrow w - 1$ ;                                /* save width */
28                     end
29                     while ( $Time_w V_i = Time_{w-1} V_i$ ) do
30                          $w \leftarrow w - 1$ ;                                /* take advantage of pareto optimal points */
31                         if ( $w = 1$ ) then
32                              $\text{break}$ ;                                /* from while */
33                         end
34                     end
35                      $Time_{V_i} \leftarrow Time_w V_i$ ;  $isTest_{V_i} \leftarrow 1$ ;  $Width_{V_i} \leftarrow w$ ;
36                      $t_{V_i} \leftarrow CurrentTstamp$ ;  $W_{available} \leftarrow W_{available} - w$ ;
37                      $RemainingWindow \leftarrow TstampSpan - Time_{V_i}$ ;
38                     if ( $RemainingWindow > 0$ ) then
39                         for  $p \leftarrow 1$  To  $n_j$  do
40                             if ( $(isTest_{V_p} = 0)$  and  $(Time_w V_p \leq RemainingWindow)$ ) then
41                                  $RemainingWindow \leftarrow RemainingWindow - Time_w V_p$ ;
42                                  $Time_{V_p} \leftarrow Time_w V_p$ ;  $isTest_{V_p} \leftarrow 1$ ;  $t_{V_p} \leftarrow Currentstamp$ ;
43                                  $Width_{V_p} \leftarrow 0$ ;                                /* No extra TAM wires required, using same TAM wire */
44                             end
45                         end
46                     end
47                     if ( $W_{available} = 0$ ) then
48                          $\text{break}$ ;                                /* out of for loop */
49                     end
50                 end
51             end
52              $TotTime \leftarrow TotTime + TotTime_j$ ;
53 end

```

cores have consumed during testing. Now as per Algorithm 1 all cores are placed initially in the TAM-partition having the width of 3, which is indicated by highlighting the corresponding test times in bold fonts. For example, if core 5 is placed in the TAM-partition of width 3, according to the bin packing method it would acquire an area of $64070 \times 3 = 192210$ square units, whereas if placed in the other TAM-partition of width 21, it requires $9989 \times 23 = 229747$ square units. Hence 192210 being the smaller of the two, core 5 is placed in the TAM-partition of width 3. So, in this way each core goes to a certain TAM-partition, however, in this example coincidentally all the cores went to the same TAM partition of width 3. Hence, after the initial placement of cores, the total test time turns out to be $(155 + 5146 + \dots + 40137) = 224874$.

Then using the *Cost* function (mentioned in Equation (1)) for $\alpha = 0.35$ (for priority on power constraint), for each core V_i , we have calculated the $Cost(V_i, Tam_j)$, as well as $Gain(V_i, Tam_j)$, when V_i is being placed in both of the TAM-partitions. In Table 3, we have shown the step wise results, which is obtained from the Algorithm 2. For example when $\alpha = 0.35$, as per Equation (1), $Cost(5, Tam_3) = (0.35 * 224874 + 0.65 * 2 * 253.25) = 79035.13$, $Gain(5, Tam_{21}) = (0.35 * 224874 + 0.65 * 2 * 253.25) - (0.35 * 160804 + 0.65 * 1 * 253.25) = 22589.11$. In Step 1 of Table 3, we find that the highest positive gain occurs if core 5 moves to the TAM-partition of width 21 (shown in bold fonts). So we place core 5 in the TAM-partition of width 21, at the end of 1st pass. Similarly, in step 2, the highest positive gain occurs if core 6 makes a movement and so on. For brevity, we do not show the intermediate steps in details. Finally, as shown in the table, in step7 we find that there is no positive gain for any of the cores, so Algorithm 2 stops. After completion of the Algorithm 2, core3, 4, 8 and 9 are placed on the TAM-partition of width 3 and rest on the TAM-partition of width 21. The cores being placed in the same TAM-partition are being tested serially, which makes the total test time $\max(31159, 31170) = 31170$. After this we have used Algorithm 3 to generate an optimal scheduling, which will allow us to test the cores parallelly and serially within their allotted TAM-partitions, which in turn gives the total test time $\max(31159, 30323) = 31159$, the details of these calculations being given in the previous subsection.

5.2. Algorithm complexity

Let K be the number of TAM partitions and $|V|$ be the number of cores in the 3D SoC. For each core Algorithm 1 performs $O(K)$ computations and as a result its time complexity is $O(K |V|)$.

The total number of passes due to the *goto* statement in Step 26 may be at most $|V|$, if we lock each core after it moves. Thus the worst-case time complexity $O(K |V|^2)$. However, in our experiments, we found that typically the number of passes required is less than $|V|/2$.

In Algorithm 3 the *while* loop in Step 12 is executed at most n_j times, i.e., the number of cores present in the TAM partition (Tam_j) of width W_j . n_j will be less than the number of cores (n) present in the SoC ($n_j \ll n$). The *for* loop in Step 14 is executed for n_j times. Each of the two *while* loops in Steps 24 and 28 may be executed at most W_j (TAM-width) times. The *while* loop at step 38 may also be executed for n_j times in the worst case. The rest of the steps can run in $O(1)$ time. This whole process is repeated for each TAM partition (Tam_j , $1 \leq j \leq K$). Thus, the worst-case time complexity of Algorithm 3 is $O(\sum_{j=1}^K n_j^2 (W_j + n_j))$, since $n_j^2 (W_j + n_j)$ clearly dominates over the time required for sorting ($n_j \log_2 n_j$) in Step 10. Finally, our proposed approach has the overall time complexity of $O(K |V| + K |V|^2 + \sum_{j=1}^K n_j^2 (W_j + n_j))$, which turns out to be $O(\sum_{j=1}^K n_j^2 (W_j + n_j))$.

5.3. An example of the architecture

Here in this section, we present an example of the required architecture to implement our scheduling algorithm (Algorithm 3) unlike the architecture that we had discussed before (Fig. 1). Let us consider Fig. 5 where Core10 and Core18 is placed at bottommost TAM partition with TAM width 32. These cores are being tested in parallel in a single

Table 3

Steps of Algorithm 2 applied on d695 for two TAM partitions.

Step-1, TAM width 3					
Core	1	2	3	4	5
Cost	78 715.27	78 726.45	78 721.17	78 785.93	79 035.13
Core	6	7	8	9	10
Cost	79 124.16	78 852.36	78 772.27	78 771.78	78 800.30
Core	1	2	3	4	5
Gain	0.0	0.0	0.0	0.0	0.0
Core	6	7	8	9	10
Gain	0.0	0.0	0.0	0.0	0.0
Step-1, TAM width 21					
Core	1	2	3	4	5
Cost	78 656.34	76 915.08	77 836.09	74 850.77	56 446.01
Core	6	7	8	9	10
Cost	57 128.23	71 093.48	75 686.73	75 658.14	64 705.15
Core	1	2	3	4	5
Gain	58.94	1811.38	885.09	3935.16	22 589.11
Core	6	7	8	9	10
Gain	21 995.93	7758.88	3085.53	3113.64	14 095.15
Step-2, TAM width 3					
Core	1	2	3	4	5
Cost	56 290.77	56 301.95	56 296.68	56 361.43	79 035.13
Core	6	7	8	9	10
Cost	56 699.66	56 427.86	56 347.77	56 347.28	56 375.80
Core	1	2	3	4	5
Gain	0.0	0.0	0.0	0.0	–22589.11
Core	6	7	8	9	10
Gain	0.0	0.0	0.0	0.0	0.0
Step-2, TAM width 21					
Core	1	2	3	4	5
Cost	56 231.84	54 490.57	55 411.59	52 426.26	56 446.01
Core	6	7	8	9	10
Cost	34 703.73	48 668.98	53 262.23	53 233.64	42 280.65
Core	1	2	3	4	5
Gain	58.94	1811.38	885.09	3935.16	0.0
Core	6	7	8	9	10
Gain	21 995.93	7758.88	3085.53	3113.64	14 095.15
...					
Step-7 (Final Step), TAM width 3					
Core	1	2	3	4	5
Cost	10 969.27	12 727.30	10 924.77	10 989.52	33 659.38
Core	6	7	8	9	10
Cost	33 110.71	18 737.76	10 975.87	10 975.38	25 048.01
Core	1	2	3	4	5
Gain	–55.09	–1807.53	0.0	0.0	–22585.26
Core	6	7	8	9	10
Gain	–21992.08	–7755.03	0.0	0.0	–14091.30
Step-7 (Final Step), TAM width 21					
Core	1	2	3	4	5
Cost	10 914.19	10 919.78	11 794.59	12 989.66	11 074.11
Core	6	7	8	9	10
Cost	11 118.63	10 982.73	12 554.43	11 438.04	10 956.70
Core	1	2	3	4	5
Gain	0.0	0.0	–869.81	–2000.14	0.0
Core	6	7	8	9	10
Gain	0.0	0.0	–1578.57	–462.66	0.0

timestamp, independent of the other TAM partition. The core18 is being tested only by using TAM width 10 and the core10 is being tested only by using TAM width 5. Note that in the Algorithm 3 as we have used the pareto optimal points total 17 TAM width kept unused as these TAM wires would not be able to reduce the test time. The other TAM partition is being tested independently in parallel with the bottommost TAM partition, and within this TAM partition the cores

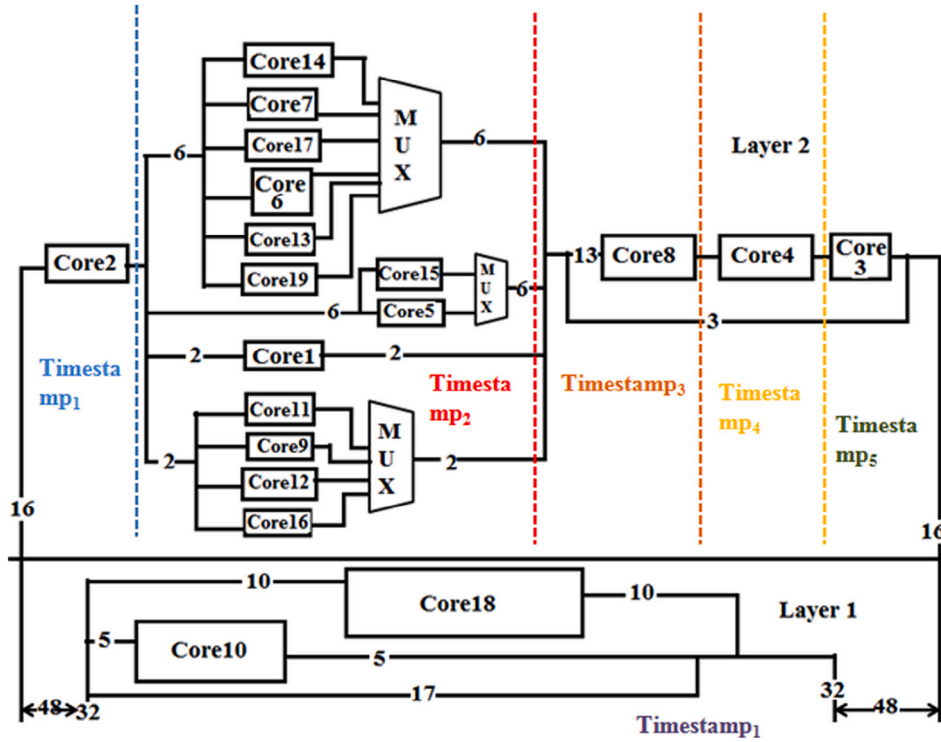


Fig. 5. Architecture for p34392 under TAM-width = 40 for Algorithm 3.

are being tested with a mix of parallel and serial scheduling, utilizing our scheduling algorithm (Algorithm 3). Groups of cores are formed which are tested in parallel but within the group, the cores are being tested serially utilizing the same TAM width at different time slots. The groups are Cores14, 7, 17, 6, 13, 19 utilizing the same TAM width 6; Cores15, 5 utilizing the same TAM width 6; Core1 using the TAM width 2; Cores11, 9, 12, 16 utilizing the same TAM width 2. All these cores are being tested in a single timestamp (*Timestamp₂*). The other cores like Core2, Core8, Core4 and Core3 are being tested serially in different timestamps.

6. Experimental results

Our proposed approach is implemented in C (Linux environment). The platform of our experiment is an Intel CoreI3 machine of the primary memory of 2 GB. Experiments were run on the ITC02 benchmarks [23]. We used the number of layers varying from 2 to 4. Compared to the previous works, the CPU time is far less even for larger benchmarks. We are unable to compare our results with the most recent work by SenGupta et al. [19] as they have not specified any kind of TAM width as well as the power threshold for the benchmarks. In another work, Kaibartta et al. [24] proposed a genetic algorithm based approach for the problem, but the power values and threshold they had used correspond to a 2D SoC. In general, the power profiling for 3D SoC requires extra care as more heated modules or cores might get stacked upon one another and as a result, the power threshold for 2D SoCs might not work for 3D SoCs.

Generally, the test time decreases monotonically with the increase of TAM width. Tables 4–7 together present the experimental results for benchmarks p93791, p34392, p22810 and d695 respectively. For the purpose of comparison, in these four tables, for the different TAM widths, we chose the TAM partitions that were used by Wu et al. [10]. The test time for each core is calculated by following the way described by Wu et al. [10] and Zorian et al. [5]. The power value for each core is obtained from a 3D SoC benchmark website [23], where they offer dynamic (switching) power values, which is generated in less

than 45 nm feature size with a 120 MHz test clock. As mentioned in their website, they do not consider static (leakage) power as it depends strongly on the temperature, which in turn depends on the environment where the benchmarks are being used and hence are difficult to predict [23].

The convex parameter α is varied from 0 to 1 with an interval of 0.05. For brevity, in these tables, we show the results only for $\alpha = 0.35$ and $\alpha = 0.80$ and highlighted by bold fonts to represent the changes over test time from our earlier work [13]. For $\alpha = 0.35$, we get the most balanced result. Wu et al. [10] and SenGupta et al. [19] did not provide any details about the thermal awareness achieved by their thermal-aware design. The rightmost column of Tables 4 to 7 gives an idea of how our algorithm is successful in bringing down the power dissipations of the upper layers. When we change α from 1 (Thermal oblivious) to 0.35 the power dissipation value of higher layers decreases at the cost of an increase in lower layers shown in those tables, which is intended. Each of the tables separately compares our experimental results for each SoC with that of Wu et al. From left to right, the columns of Tables 4 to 7 successively present the number of layers, TAM-partitions, TSV-limits, 3D test time and CPU time.

These parameters when compared with Wu et al. [10] and our earlier work [13], the results clearly establish the efficacy of our proposed algorithm. We do not show results by Pradhan et al. [11] since they did only bi-partitions also their approach was not thermal aware. Compared to earlier works in this work, our CPU times are far less even for larger benchmarks. Our algorithm has reported better (in 47.02% cases) test times or at least as good compared to one of our earlier works [13] and always better than Wu et al. [10]. Note that the number of TSVs reported is also less, even for larger benchmarks, which is very encouraging as TSVs are quite expensive. With the variation of the width of the TAM partitions the test time and the number of TSVs will also vary, for which we might get an optimal partition of TAM width that ensures lesser test time as well as lesser number of TSVs. We do not show any result by varying the width of the TAM partitions, we have considered the widths that are considered by Wu et al.

The test-time and TSV-Count for an entire Soc is totally dependent on how we have partitioned TAM wires. Exactly the same thing is

Table 4
Experimental results of multi-partitioning on p93791 (32 cores).

$\alpha=0.35$													
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption			
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2	Layer3	Layer4
2	16 (3, 13)	6	80	1911 161	1911 161	2103 700	0.0	0.0	8.2	11 351.52	3026.88		
2	24 (4, 20)	8	80	1 408 153	1 408 153	1 406 320	0.0	0.0	22.0	11 351.52	3026.88		
2	32 (5, 27)	10	80	1 008 607	1 008 607	1 405 440	0.0	0.0	160.6	11 998.91	2379.49		
2	40 (5, 35)	10	80	992 636	992 751	1 241 245	0.0	0.0	160.6	12 004.49	2373.91		
2	48 (5, 43)	10	80	888 792	888 792	1 405 440	0.0	0.0	160.6	11 999.77	2378.63		
2	56 (5, 51)	10	80	573 720	573 720	1 152 364	0.0	0.0	160.6	13 919.13	459.27		
2	64 (5, 59)	10	80	560 442	573 720	1 008 220	0.0	0.0	160.6	13 919.13	459.27		
										Layer1	Layer2	Layer3	
3	16 (1, 1, 14)	6	80	2 005 814	2 005 814	2 229 152	0.01	0.01	1.9	12 106.10	1894.39	377.90	
3	24 (1, 4, 19)	12	80	1 412 113	1 412 113	1 412 113	0.00	0.00	3.5	13 523.72	839.50	15.18	
3	32 (1, 7, 24)	18	80	946 604	946 604	1 009 262	0.00	0.00	6.9	7979.0	6380.78	18.62	
3	40 (1, 16, 23)	36	80	733 563	734 000	999 931	0.00	0.00	53.8	5853.65	8184.47	340.28	
3	48 (7, 16, 25)	60	80	641 170	641 170	995 650	0.00	0.00	96.3	4769.35	9103.81	505.23	
3	56 (13, 19, 24)	90	80	561 893	561 893	995 522	0.00	0.00	302.8	6070.66	7088.46	1219.27	
3	64 (4, 25, 35)	66	80	575 005	575 005	995 522	0.00	0.00	46.6	4907.93	9124.77	345.70	
										Layer1	Layer2	Layer3	Layer4
4	16 (1, 1, 2, 12)	14	80	1 946 902	1 946 902	4 023 112	0.02	0.02	2.2	12 973.79	1284.29	13.12	107.19
4	24 (1, 1, 1, 21)	12	80	1 503 815	1 503 815	3 823 747	0.01	0.01	7.0	13 378.07	351.22	0.86	648.25
4	32 (1, 1, 4, 26)	18	80	1 023 723	1 023 723	3 695 454	0.03	0.03	2.0	12 997.63	1037.45	343.32	0.86
4	40 (1, 10, 1, 28)	30	80	846 151	846 151	2 699 795	0.01	0.01	55.4	7429.93	6932.43	15.18	0.86
4	48 (4, 13, 1, 30)	48	80	731 269	731 269	1 903 700	0.03	0.03	220.5	6269.04	6454.94	1651.22	3.20
4	56 (4, 1, 25, 26)	72	80	557 484	557 484	1 121 150	0.03	0.03	252.2	6070.66	7587.97	706.01	13.76
4	64 (1, 10, 25, 28)	96	80	509 566	509 566	995 522	0.05	0.05	339.7	4527.49	8743.96	1101.21	5.73
$\alpha=0.80$													
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption			
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2	Layer3	Layer4
2	16 (3, 13)	6	80	1911 161	1911 161	2103 700	0.0	0.0	8.2	11 351.52	3026.88		
2	24 (4, 20)	8	80	1 408 153	1 408 153	1 406 320	0.0	0.0	22.0	11 351.52	3026.88		
2	32 (5, 27)	10	80	1 008 607	1 008 607	1 405 440	0.0	0.0	160.6	11 998.91	2379.49		
2	40 (5, 35)	10	80	992 751	992 751	1 241 245	0.00	0.00	160.6	12 004.49	2373.91		
2	48 (5, 43)	10	80	888 792	888 792	1 405 440	0.0	0.0	160.6	11 999.77	2378.63		
2	56 (5, 51)	10	80	573 720	573 720	1 152 364	0.0	0.0	160.6	13 919.13	459.27		
2	64 (5, 59)	10	80	573 720	573 720	1 008 220	0.0	0.0	160.6	13 919.13	459.27		
										Layer1	Layer2	Layer3	
3	16 (1, 1, 14)	6	80	2 005 814	2 005 814	2 229 152	0.01	0.01	1.9	12 106.10	1894.39	377.90	
3	24 (1, 4, 19)	12	80	1 410 725	1 410 725	1 412 113	0.00	0.00	3.5	12 087.74	2268.85	21.81	
3	32 (1, 7, 24)	18	80	946 604	946 604	1 009 262	0.00	0.00	6.9	7979.0	6372.88	26.55	
3	40 (1, 16, 23)	36	80	742 629	742 629	999 931	0.00	0.00	53.8	5856.80	8499.79	21.81	
3	48 (7, 16, 25)	60	80	641 170	641 170	995 650	0.00	0.00	96.3	4769.35	9097.43	511.61	
3	56 (13, 19, 24)	90	80	561 893	561 893	995 522	0.00	0.00	302.8	6070.66	7079.76	1227.97	
3	64 (4, 25, 35)	66	80	575 005	575 005	995 522	0.00	0.00	46.6	4907.93	9118.39	352.08	
										Layer1	Layer2	Layer3	Layer4
4	16 (1, 1, 2, 12)	14	80	1 946 445	1 946 445	4 023 112	0.02	0.02	2.2	13 523.72	724.81	22.67	107.19
4	24 (1, 1, 1, 21)	12	80	1 503 815	1 503 815	3 823 747	0.01	0.01	7.0	13 378.07	351.22	648.25	0.86
4	32 (1, 1, 4, 26)	18	80	1 023 723	1 023 723	3 695 454	0.03	0.03	2.0	12 997.63	1029.55	351.22	0.00
4	40 (1, 10, 1, 28)	30	80	846 151	846 151	2 699 795	0.01	0.01	55.4	7429.93	6921.09	26.52	0.86
4	48 (4, 13, 1, 30)	48	80	715 686	715 686	1 903 700	0.02	0.02	220.5	7054.31	6767.43	534.85	21.81
4	56 (4, 1, 25, 26)	72	80	563 763	563 763	1 121 150	0.03	0.03	252.2	6080.44	7903.31	380.19	14.45
4	64 (1, 10, 25, 28)	96	80	502 275	502 275	995 522	0.05	0.05	339.7	5747.11	7479.13	1136.91	15.25

Table 5
Experimental results of multi-partitioning on p34392 (19 cores).

$\alpha=0.35$													
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption			
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2	Layer3	Layer4
2	16 (8, 8)	16	70	1 091 810	1 091 810	1 081 570	0.0	0.0	1.9	36 428.5		80 748.21	
2	24 (9, 15)	18	70	859 758	859 758	1 064 994	0.0	0.0	5.5	89 581.71		27 595.01	
2	32 (15, 17)	30	70	551 778	762 419	831 177	0.0	0.0	6.7	60 221.42		56 955.30	
2	40 (9, 31)	18	70	699 807	845 337	831 177	0.0	0.0	7.0	89 746.11		27 430.61	
2	48 (16, 32)	32	70	544 579	781 178	808 844	0.0	0.0	5.7	80 717.68		36 459.03	
2	56 (16, 40)	32	70	544 579	781 178	808 844	0.0	0.0	5.6	80 717.68		36 459.03	
2	64 (23, 41)	46	70	544 579	721 072	808 844	0.0	0.0	5.6	80 717.68		36 459.03	
											Layer1	Layer2	Layer3
3	16 (1, 7, 8)	18	70	1 250 802	1 250 802	1 005 088	0.00	0.00	1.0	89 581.1	27 593	2.014	
3	24 (1, 10, 13)	24	70	1 128 588	1 128 588	1 049 435	0.00	0.00	2.6	89 581.1	27 583.8	11.12	
3	32 (10, 7, 15)	48	70	859 758	859 758	933 906	0.00	0.00	10.8	89 581.71	27 594.0	1.002	
3	40 (7, 16, 17)	60	70	544 579	570 385	859 998	0.00	0.00	4.8	60 031.17	51 060.74	6084.81	
3	48 (7, 13, 28)	54	70	544 579	838 643	843 401	0.00	0.00	34.5	89 581.71	27 593	2.0018	
3	56 (10, 13, 33)	66	70	773 222	781 178	832 498	0.00	0.00	19.6	80 717.68	30 328.95	6130.08	
3	64 (10, 13, 41)	66	70	773 222	781 178	689 711	0.00	0.00	2.2	80 717.68	30 328.95	6130.08	
											Layer1	Layer2	Layer3
4	16 (1, 1, 1, 13)	12	70	1 365 187	1 365 187	1 391 607	0.00	0.00	3.8	110 633.96	6335.58	30.52	176.65
4	24 (1, 1, 1, 21)	12	70	838 643	1 124 588	1 391 607	0.00	0.00	0.8	110 809.61	6244.36	92.22	30.52
4	32 (1, 1, 7, 23)	24	70	838 643	838 643	834 826	0.01	0.01	7.0	89 581.71	27 381.59	37.76	175.65
4	40 (1, 10, 1, 28)	30	70	622 969	838 643	790 995	0.00	0.00	45.5	89 581.71	27 583.89	11.12	1.01
4	48 (1, 13, 13, 21)	84	70	584 009	584 300	754 307	0.01	0.01	26.5	59 757.66	27 276.3	29 916.27	226.48
4	56 (7, 13, 10, 26)	108	70	584 524	584 524	661 844	0.03	0.03	225.3	60 091.98	27 106.31	29 916.27	62.14
4	64 (7, 13, 10, 34)	108	70	584 524	584 524	661 844	0.03	0.03	201.7	60 011.54	27 218.15	29 916.25	30.74
$\alpha=0.80$													
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption			
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2	Layer3	Layer4
2	16 (8, 8)	16	70	1 091 810	1 091 810	1 081 570	0.0	0.0	1.9	36 428.5		80 748.21	
2	24 (9, 15)	18	70	859 758	859 758	1 064 994	0.0	0.0	5.5	89 581.71		27 595.01	
2	32 (15, 17)	30	70	551 778	762 419	831 177	0.0	0.0	6.7	60 221.42		56 955.30	
2	40 (9, 31)	18	70	699 807	845 337	831 177	0.0	0.0	7.0	89 746.11		27 430.61	
2	48 (16, 32)	32	70	544 579	781 178	808 844	0.0	0.0	5.7	80 717.68		36 459.03	
2	56 (16, 40)	32	70	544 579	781 178	808 844	0.0	0.0	5.6	80 717.68		36 459.03	
2	64 (23, 41)	46	70	544 579	781 178	808 844	0.0	0.0	5.6	80 717.68		36 459.03	
											Layer1	Layer2	Layer3
3	16 (1, 7, 8)	18	70	1 250 802	1 250 802	1 005 088	0.00	0.00	1.0	89 581.7	27 588.65	6.36	
3	24 (1, 10, 13)	24	70	1 128 588	1 128 588	1 049 435	0.00	0.00	2.6	89 581.1	27 583.8	11.12	
3	32 (10, 7, 15)	48	70	859 758	859 758	933 906	0.00	0.00	10.8	89 581.71	27 594.0	1.002	
3	40 (7, 16, 17)	60	70	544 579	570 385	859 998	0.00	0.00	4.8	60 031.17	51 060.74	6084.81	
3	48 (7, 13, 28)	54	70	544 579	838 643	843 401	0.00	0.00	34.5	89 581.71	27 593	2.0018	
3	56 (10, 13, 33)	66	70	773 222	781 178	832 498	0.00	0.00	19.6	80 717.68	30 328.95	6130.08	
3	64 (10, 13, 41)	66	70	773 222	781 178	689 711	0.00	0.00	2.2	80 717.68	30 328.95	6130.08	
											Layer1	Layer2	Layer3
4	16 (1, 1, 1, 13)	12	70	1 365 187	1 365 187	1 391 607	0.00	0.00	3.8	110 633.96	6335.58	30.52	176.65
4	24 (1, 1, 1, 21)	12	70	838 643	1 124 588	1 391 607	0.00	0.00	0.8	110 809.61	6244.36	92.22	30.52
4	32 (1, 1, 7, 23)	24	70	838 643	838 643	834 826	0.01	0.01	7.0	89 581.71	27 381.59	37.76	175.65
4	40 (1, 10, 1, 28)	30	70	622 969	838 643	790 995	0.00	0.00	45.5	89 581.71	27 583.89	11.12	1.01
4	48 (1, 13, 13, 21)	84	70	584 009	584 300	754 307	0.01	0.01	26.5	59 757.66	27 276.3	29 916.27	226.48
4	56 (7, 13, 10, 26)	108	70	584 524	584 524	661 844	0.03	0.03	225.3	60 091.98	27 106.31	29 916.27	62.14
4	64 (7, 13, 10, 34)	108	70	584 524	584 524	661 844	0.03	0.03	201.7	60 011.54	27 218.15	29 916.25	30.74

Table 6
Experimental results of multi-partitioning on p22810 (28 cores).

$\alpha=0.35$													
Layer	TAM-width	TSV-Limit		3D test time			CPU time			Power unit consumption			
	(Partitions)	Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2		
2	16 (6, 10)	12	80	468 975	468 975	324 902	0.0	0.0	0.2	3635.02	5671.39		
2	24 (11, 13)	22	80	342 957	372 515	308 674	0.0	0.0	1.1	4510.98	4795.43		
2	32 (15, 17)	30	80	268 454	330 499	270 379	0.0	0.0	0.3	2793.70	6512.72		
2	40 (19, 21)	38	80	248 382	322 209	265 452	0.0	0.0	0.7	2880.77	6425.65		
2	48 (19, 29)	38	80	211 752	301 410	265 452	0.0	0.0	0.5	5452.36	3854.06		
2	56 (19, 37)	38	80	182 315	284 734	245 234	0.0	0.0	0.3	6087.31	3219.11		
2	64 (19, 45)	38	80	182 315	284 734	245 234	0.0	0.0	0.7	6087.31	3219.11		
										Layer1	Layer2	Layer3	
3	16 (4, 4, 8)	24	80	483 938	483 938	321 172	0.15	0.15	1.6	5354.54	3944.75	7.13	
3	24 (1, 10, 13)	24	80	359 473	359 473	282 849	0.0	0.0	1.6	5999.21	3126.42	180.79	
3	32 (6, 10, 16)	44	80	277 486	279 533	258 978	0.0	0.0	38.9	5348.45	2570.22	1387.75	
3	40 (10, 13, 17)	66	80	234 455	236 834	230 462	0.0	0.0	47.8	4448.84	3476.26	1381.32	
3	48 (10, 13, 25)	66	80	234 455	234 455	226 844	0.0	0.0	37.4	4858.69	3456.06	991.67	
3	56 (10, 13, 33)	66	80	183 259	215 257	226 813	0.0	0.0	17.6	3598.60	3949.03	1758.79	
3	64 (10, 13, 41)	66	80	183 259	215 257	223 079	0.0	0.0	6.5	3598.60	3949.03	1758.79	
										Layer1	Layer2	Layer3	Layer4
4	16 (1, 1, 1, 13)	12	80	542 917	542 917	309 905	0.03	0.03	0.4	7169.70	877.20	524.97	734.56
4	24 (1, 4, 4, 15)	30	80	350 630	350 712	271 146	0.05	0.05	4.2	6526.72	1227.06	1324.05	228.59
4	32 (1, 4, 13, 14)	48	80	284 805	284 805	240 120	0.04	0.04	45.5	4637.06	2546.75	1941.82	180.8
4	40 (3, 7, 13, 17)	72	80	236 751	236 977	220 194	0.03	0.03	73.5	4545.04	2655.77	1689.28	416.33
4	48 (4, 10, 13, 21)	90	80	210 448	219 394	214 379	0.03	0.03	297.5	4493.37	2216.57	2412.82	183.66
4	56 (4, 10, 13, 29)	90	80	188 696	188 696	189 146	0.04	0.04	315	4409.98	2279.40	2016.44	600.60
4	64 (1, 10, 13, 40)	72	80	182 728	201 571	181 698	0.08	0.08	407	5371.11	2565.90	1299.52	69.90
$\alpha=0.80$													
Layer	TAM-width	TSV-Limit		3D test time			CPU time			Power unit consumption			
	(Partitions)	Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2		
2	16 (6, 10)	12	80	468 975	468 975	324 902	0.0	0.0	0.2	3635.02	5671.39		
2	24 (11, 13)	22	80	342 957	372 515	308 674	0.0	0.0	1.1	4510.98	4795.43		
2	32 (15, 17)	30	80	268 454	330 499	270 379	0.0	0.0	0.3	2793.70	6512.72		
2	40 (19, 21)	38	80	248 382	322 209	265 452	0.0	0.0	0.7	2880.77	6425.65		
2	48 (19, 29)	38	80	211 752	301 410	265 452	0.0	0.0	0.5	5452.36	3854.06		
2	56 (19, 37)	38	80	182 315	284 734	245 234	0.0	0.0	0.3	6087.31	3219.11		
2	64 (19, 45)	38	80	182 315	284 734	245 234	0.0	0.0	0.7	6087.31	3219.11		
										Layer1	Layer2	Layer3	
3	16 (4, 4, 8)	24	80	462 450	470 998	321 172	0.15	0.15	1.6	2681.90	3384.96	3239.54	
3	24 (1, 10, 13)	24	80	359 473	359 473	282 849	0.0	0.0	1.6	5999.21	3126.42	180.79	
3	32 (6, 10, 16)	44	80	272 775	279 429	258 978	0.0	0.0	38.9	5332.25	2536.10	1438.07	
3	40 (10, 13, 17)	66	80	234 455	236 834	230 462	0.0	0.0	47.8	4448.84	3476.26	1381.32	
3	48 (10, 13, 25)	66	80	234 455	234 455	226 844	0.0	0.0	37.4	4858.69	3456.06	991.67	
3	56 (10, 13, 33)	66	80	183 259	215 257	226 813	0.0	0.0	17.6	3598.60	3949.03	1758.79	
3	64 (10, 13, 41)	66	80	183 259	215 257	223 079	0.0	0.0	6.5	3598.60	3949.03	1758.79	
										Layer1	Layer2	Layer3	Layer4
4	16 (1, 1, 1, 13)	12	80	542 917	542 917	309 905	0.03	0.03	0.4	7169.70	877.20	524.97	734.56
4	24 (1, 4, 4, 15)	30	80	350 630	350 712	271 146	0.05	0.05	4.2	6569.75	1261.56	1446.52	228.59
4	32 (1, 4, 13, 14)	48	80	277 553	289 753	240 120	0.04	0.04	45.5	4417.11	2948.47	1931.63	9.21
4	40 (3, 7, 13, 17)	72	80	236 751	236 977	220 194	0.03	0.03	73.5	4545.04	2655.77	1689.28	416.33
4	48 (4, 10, 13, 21)	90	80	210 448	219 394	214 379	0.03	0.03	297.5	4493.37	2216.57	2412.82	183.66
4	56 (4, 10, 13, 29)	90	80	188 696	188 696	189 146	0.04	0.04	315	4409.98	2279.40	2016.44	600.60
4	64 (1, 10, 13, 40)	72	80	182 728	201 571	181 698	0.08	0.08	407	5371.11	2565.90	1299.52	69.90

Table 7
Experimental results of multi-partitioning on *d695* (10 cores).

$\alpha=0.35$											
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption	
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2
2	16 (5, 11)	10	60	47 691	47 691	44 815	0.0	0.0	0.8	647.61	310.71
2	24 (3, 21)	6	60	31 159	31 170	31 170	0.0	0.0	0.2	783.29	175.04
2	32 (16, 16)	32	60	25 829	25 829	26 578	0.0	0.0	0.4	332.65	625.66
2	40 (20, 20)	40	60	22 365	23 247	22 848	0.0	0.0	0.6	355.77	602.54
2	48 (16, 32)	32	60	19 938	19 938	20 314	0.0	0.0	0.9	647.61	310.72
2	56 (24, 32)	48	60	19 938	19 938	19 457	0.0	0.0	0.9	647.61	310.72
2	64 (32, 32)	38	60	16 075	19 080	19 457	0.0	0.0	0.8	364.38	593.35
										Layer1	Layer2
3	16 (2, 5, 9)	18	60	42 424	42 424	44 809	0.0	0.0	1.3	574.99	208.30
3	24 (1, 2, 21)	8	60	31 739	31 739	31 132	0.0	0.0	7.5	810.95	131.57
3	32 (4, 9, 19)	34	60	21 566	21 566	21 856	0.0	0.0	46.8	598.01	185.28
3	40 (5, 9, 26)	38	60	21 527	21 527	21 005	0.0	0.0	10.9	632.88	185.28
3	48 (4, 12, 32)	40	60	16 454	19 434	18 799	0.0	0.0	2.0	687.65	192.07
3	56 (5, 17, 34)	54	60	15 634	15 634	13 575	0.0	0.0	1.3	574.99	258.98
3	64 (7, 17, 40)	62	60	15 300	15 311	19 457	0.0	0.0	1.3	403.25	434.39
										Layer1	Layer2
4	16 (1, 1, 5, 9)	20	60	43 193	43 193	44 809	0.0	0.0	14.5	445.74	383.30
4	24 (1, 1, 1, 21)	12	60	31 739	31 739	31 132	0.0	0.0	53.8	810.95	70.01
4	32 (1, 2, 10, 19)	34	60	22 174	22 427	22 427	0.1	0.1	20.0	564.91	269.06
4	40 (1, 4, 9, 26)	32	60	26 123	26 163	21 005	0.0	0.0	7.6	568.58	304.30
4	48 (2, 2, 12, 32)	56	60	18 350	19 845	18 799	0.1	0.1	43.4	515.90	328.95
4	56 (1, 4, 17, 34)	56	60	12 987	13 785	13 575	0.0	0.0	13.8	489.21	344.76
4	64 (3, 6, 16, 39)	74	60	14 974	15 074	12 841	0.0	0.0	14.2	582.20	184.86
$\alpha=0.80$											
Layer	TAM-width (Partitions)	TSV-Limit		3D test time			CPU time			Power unit consumption	
		Proposed	[10]	Proposed	[13]	[10]	Proposed	[13]	[10]	Layer1	Layer2
2	16 (5, 11)	10	60	47 691	47 691	44 815	0.0	0.0	0.8	647.61	310.71
2	24 (3, 21)	6	60	31 159	31 170	31 170	0.0	0.0	0.2	783.29	175.04
2	32 (16, 16)	32	60	25 829	27 258	26 578	0.0	0.0	0.4	332.65	625.66
2	40 (20, 20)	40	60	22 365	23 247	22 848	0.0	0.0	0.6	355.77	602.54
2	48 (16, 32)	32	60	19 938	19 938	20 314	0.0	0.0	0.9	647.61	310.72
2	56 (24, 32)	48	60	19 938	19 938	19 457	0.0	0.0	0.9	647.61	310.72
2	64 (32, 32)	38	60	16 075	19 080	19 457	0.0	0.0	0.8	364.38	593.35
										Layer1	Layer2
3	16 (2, 5, 9)	18	60	42 424	42 424	44 809	0.0	0.0	1.3	574.99	208.30
3	24 (1, 2, 21)	8	60	31 739	31 739	31 132	0.0	0.0	7.5	810.95	131.57
3	32 (4, 9, 19)	34	60	21 566	21 566	21 856	0.0	0.0	46.8	598.01	185.28
3	40 (5, 9, 26)	38	60	21 527	21 527	21 005	0.0	0.0	10.9	632.88	185.28
3	48 (4, 12, 32)	40	60	16 454	19 434	18 799	0.0	0.0	2.0	687.65	192.07
3	56 (5, 17, 34)	54	60	15 634	15 634	13 575	0.0	0.0	1.3	574.99	258.98
3	64 (7, 17, 40)	62	60	15 300	15 311	19 457	0.0	0.0	1.3	403.25	434.39
										Layer1	Layer2
4	16 (1, 1, 5, 9)	20	60	43 193	43 193	44 809	0.0	0.0	14.5	445.74	383.30
4	24 (1, 1, 1, 21)	12	60	31 739	31 739	31 132	0.0	0.0	53.8	810.95	70.01
4	32 (1, 2, 10, 19)	34	60	22 174	22 427	22 427	0.1	0.1	20.0	564.91	269.06
4	40 (1, 4, 9, 26)	32	60	26 123	26 163	21 005	0.0	0.0	7.6	568.58	304.30
4	48 (2, 2, 12, 32)	56	60	18 350	19 845	18 799	0.1	0.1	43.4	515.90	328.95
4	56 (1, 4, 17, 34)	56	60	12 987	13 785	13 575	0.0	0.0	13.8	489.21	344.76
4	64 (3, 6, 16, 39)	74	60	14 974	15 074	12 841	0.0	0.0	14.2	582.20	184.86

Table 8

Experiment on varying TAM-partitions for No. of layers = 2.

$\alpha=0.35$		Benchmark : p93791		
TAM partition	TSV-count	3D test time	Power unit consumption	
			Layer 1	Layer 2
4, 20	8	1 408 153	11 351.52	3026.88
10, 14	20	1 317 204	9065.01	5313.39
$\alpha=0.35$		Benchmark : p34392		
TAM partition	TSV-count	3D test time	Power unit consumption	
			Layer 1	Layer 2
9, 15	18	859 758	89 581.71	27 595.01
10, 14	20	1 128 073	89 581.71	27 595.01
$\alpha=0.35$		Benchmark : p22810		
TAM partition	TSV-count	3D test time	Power unit consumption	
			Layer 1	Layer 2
11, 13	22	342 957	4510.98	4795.43
10, 14	20	365 287	4272.69	5033.73
$\alpha=0.35$		Benchmark : d695		
TAM partition	TSV-count	3D test time	Power unit consumption	
			Layer 1	Layer 2
3, 21	6	31 159	783.29	175.04
10, 14	20	35 555	625.67	332.66

Table 9

Experiment on varying TAM-partitions for No. of layers = 3.

$\alpha=0.35$		Benchmark : p93791			
TAM partition	TSV-count	3D test time	Power unit consumption		
			Layer 1	Layer 2	Layer 3
1, 4, 19	12	1 412 113	13 523.72	839.5	15.18
6, 8, 10	40	1 243 752	3472.38	8787.81	2118.21
$\alpha=0.35$		Benchmark : p34392			
TAM partition	TSV-count	3D test time	Power unit consumption		
			Layer 1	Layer 2	Layer 3
1, 10, 13	24	1 128 588	89 581.1	27 583.8	11.12
6, 8, 10	40	792 553	80 717.68	36 062.39	396.65
$\alpha=0.35$		Benchmark : p22810			
TAM partition	TSV-count	3D test time	Power unit consumption		
			Layer 1	Layer 2	Layer 3
1, 10, 13	24	359 473	5999.21	3126.42	180.79
6, 8, 10	40	323 885	4581.47	2045.58	2679.37
$\alpha=0.35$		Benchmark : d695			
TAM partition	TSV-count	3D test time	Power unit consumption		
			Layer 1	Layer 2	Layer 3
1, 2, 21	8	31 739	810.95	131.57	15.81
6, 8, 10	40	32 167	469.17	235.91	253.25

being reflected in the tables from Tables 8–10, where we have divided TAM wire of width 24 into TAM-partitions of different width selected randomly (i.e. in 2 TAM-partitions of TAM-width 10 and 14, 3 TAM-partitions of TAM-width 6, 8 and 10 and 4 TAM-partitions of TAM-width 2, 5, 5 and 9 for 2, 3 and 4 number of layers respectively) for all four benchmarks that we have used. Again this case study is conducted for $\alpha = 0.35$. In Table 10 the number of TSVs is counted as $(2 + 5 + 8) \times 2 + (2 + 5) \times 2 + 2 \times 2 = 48$. This is because as the TAM-partition with TAM width 9 is going to be placed at the bottommost layer. Similarly, TAM-partitions of TAM width 8, 5, 2 is placed at layer 2, 3 and 4 respectively.

We have extended our experiments to find the best TAM partitions for different benchmarks under different criteria. We report the results in Tables 11–13. The first four columns of these tables show the result of one benchmark and the other four columns for another benchmark.

Table 10

Experiment on varying TAM-partitions for No. of layers = 4.

$\alpha=0.35$		Benchmark : p93791				
TAM partition	TSV-count	3D test time	Power unit consumption			
			Layer 1	Layer 2	Layer 3	Layer 4
1, 1, 1, 21	12	1 503 815	13 378.07	351.22	0.86	648.25
2, 5, 8, 9	48	1 255 020	3962.59	4250.95	6048.22	116.63
$\alpha=0.35$		Benchmark : p34392				
TAM Partition	TSV-count	3D test time	Power unit consumption			
			Layer 1	Layer 2	Layer 3	Layer 4
1, 1, 1, 21	12	838 643	110 809.61	6244.36	92.22	30.52
2, 5, 8, 9	48	939 855	65 970.27	21 290.17	29 916.27	1.52
$\alpha=0.35$		Benchmark : p22810				
TAM partition	TSV-count	3D test time	Power unit consumption			
			Layer 1	Layer 2	Layer 3	Layer 4
1, 4, 4, 15	30	350 630	6526.72	1227.06	1324.05	228.59
2, 5, 8, 9	48	377 886	5401.5	3028.25	876.66	1.57
$\alpha=0.35$		Benchmark : d695				
TAM partition	TSV-count	3D test time	Power unit consumption			
			Layer 1	Layer 2	Layer 3	Layer 4
1, 1, 1, 21	12	31 739	810.95	70.01	15.81	61.56
2, 5, 8, 9	48	37 364	474.84	161.73	321.73	2.56

In each of these tables, we also present the different TAM partitions that had been used by Wu et al. [10] and their corresponding test time under the same criterion of total TAM-width used. Table 11 presents the result emphasized on power dissipation and Tables 12 and 13 presents the result emphasized on test time. Note that in all the benchmarks except for the thermal oblivious case of d695 (Table 13), our test-times are considerably less in almost all cases. The charts shown in Figs. 6–8, make this more evident. There are some interesting observations for benchmark p22810, where the width of the four TAM partitions are same when $\alpha = 0.35$, $\alpha = 0.8$ and total TAM width is 32, 40, 48, but their corresponding test times are different. Similar cases can be found in these tables. Another observation that is counter-intuitive is that in some cases, we have obtained better test time while emphasizing on power dissipation rather than doing so on test time. As for example, for the benchmark p34392 under total TAM width 24, for three TAM partitions we get better test time when $\alpha = 0.35$ than when $\alpha = 1.0$. This reflects the fact that test time depends on various factors such as how Algorithm 2 allocates cores in different TAM partitions, and how Algorithm 3 schedules the cores to be tested.

We also compare our results with a more recent work done by Kaibartta et al. [24] in Table 14, for the two benchmarks that they had used. Since the power considerations in the two setups are very different, we have compared the results for only the thermal oblivious cases. Note that the corresponding TAM partitions are different in some cases but the TAM widths used are kept same for the purpose of comparison between the test-times. From the plots shown in Fig. 9, it is apparent that here also our test-times are in general better than their ones.

7. Conclusion and future work

We have proposed three algorithms for core distributing and test scheduling related to the design of TAM-architecture of 3D-SoCs that lead to reduced test time while maintaining the constraints on TSV-count and test-power dissipation. Experimental results show that these algorithms require an insignificant amount of CPU-time. The proposed approach may thus be useful in reducing the production cost of SoCs as fewer test-TSVs and thermal-TSVs are required.

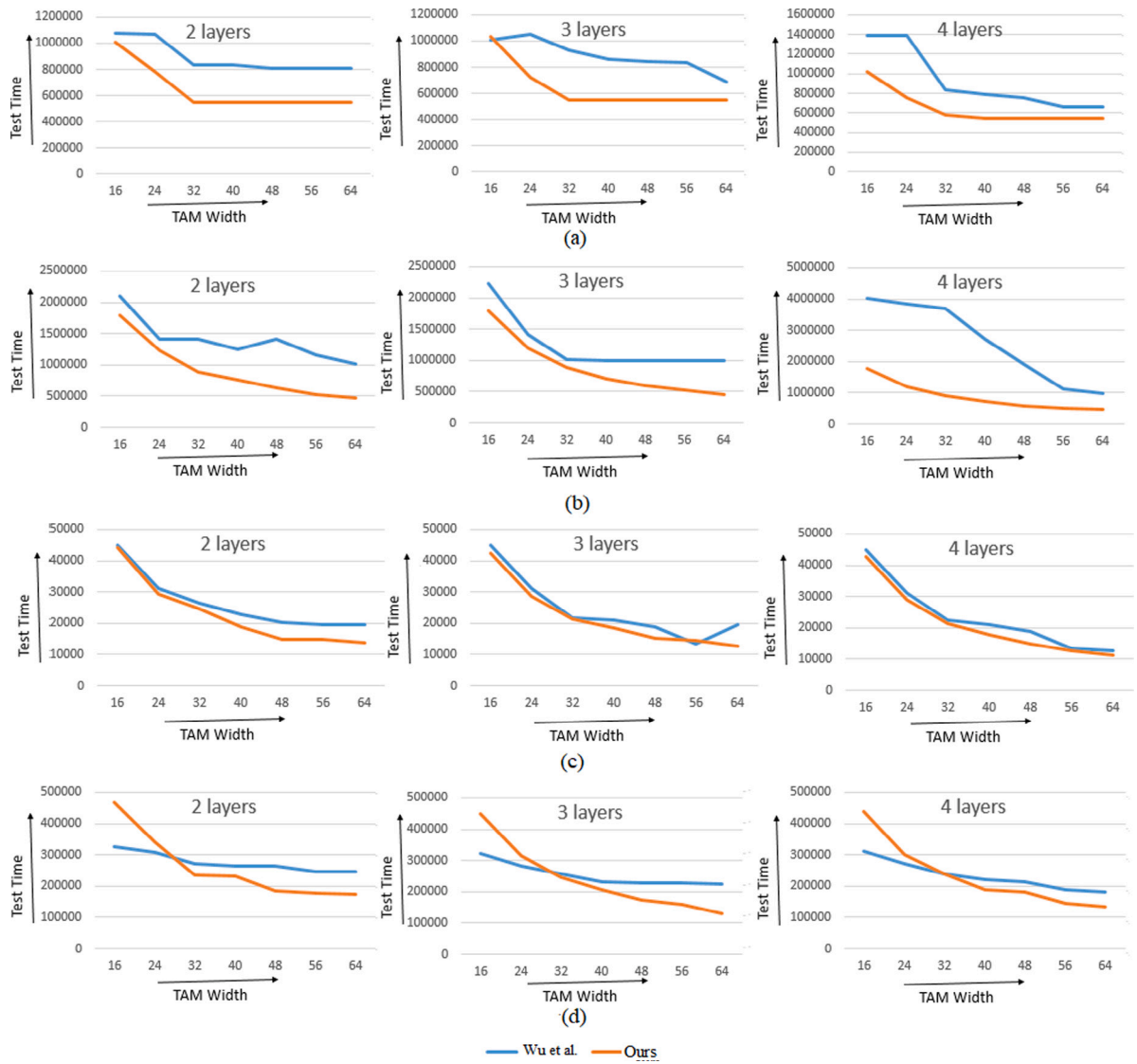


Fig. 6. Summarized result for Table 11 (a) result for p34392 (b) result for p93791 (c) result for d695 (d) result for p22810.

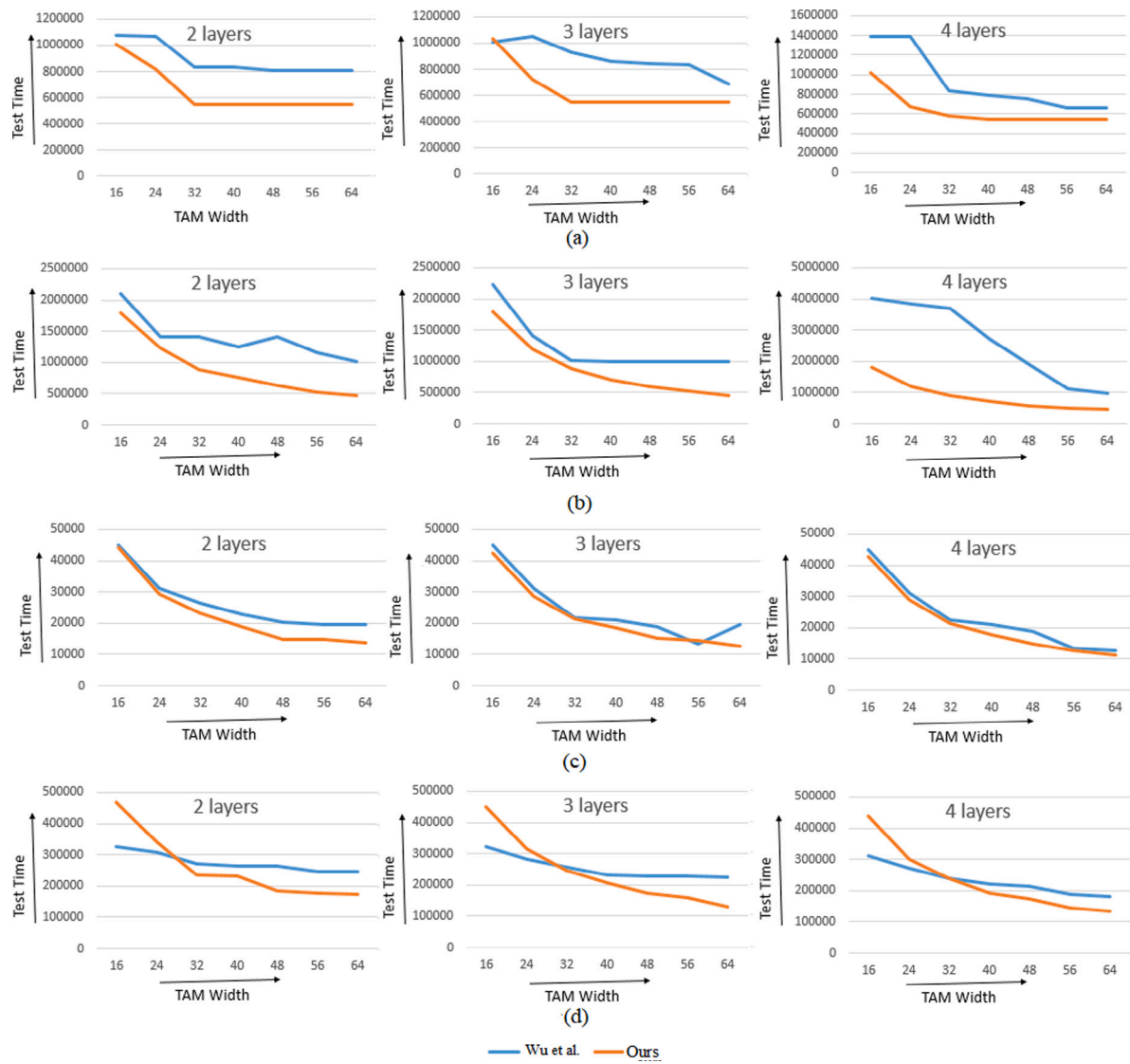


Fig. 7. Summarized result for Table 12 (a) result for p34392 (b) result for p93791 (c) result for d695 (d) result for p22810.

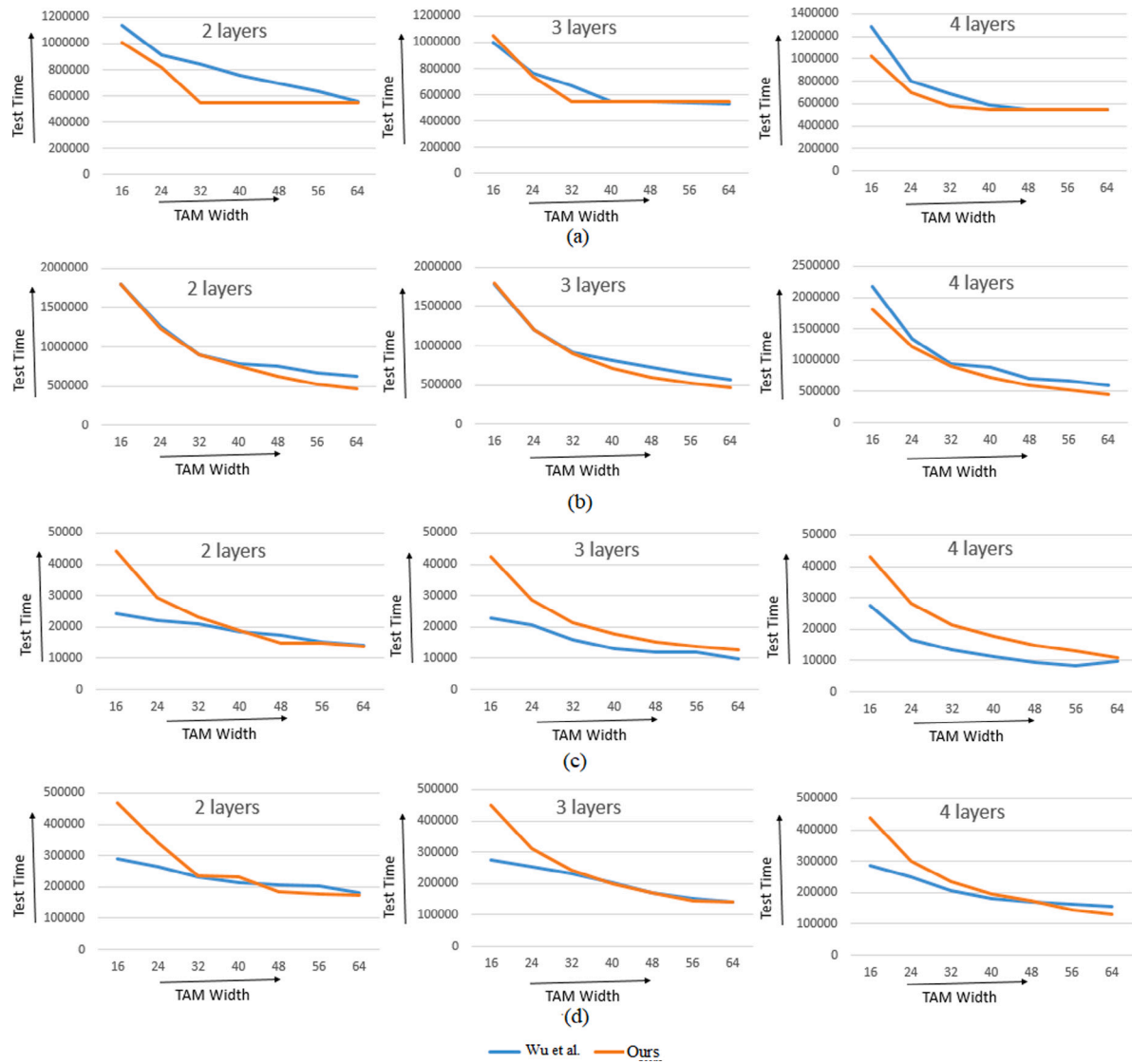


Fig. 8. Summarized result for Table 13 (a) result for p34392 (b) result for p93791 (c) result for d695 (d) result for p22810.

Table 11Test time under $\alpha=0.35$, Maximum priority on power dissipation.

$\alpha=0.35$							
p34392				p93791			
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
8, 8	1081570	1, 15	1007794	3, 13	2103700	7, 9	1800390
1, 7, 8	1005088	5, 5, 6	1031703	1, 1, 14	2229152	3, 5, 8	1797898
1, 1, 1, 13	1391607	1, 5, 5, 5	1023511	1, 1, 2, 12	4023112	3, 4, 4, 5	1789364
Total TAM width = 24							
9, 15	1064994	11, 13	781178	4, 20	1406320	1, 23	1230138
1, 10, 13	1049435	7, 8, 9	720854	1, 4, 19	1412113	7, 8, 9	1192496
1, 1, 1, 21	1391607	3, 5, 8, 8	757885	1, 1, 1, 21	3823747	5, 5, 6, 8	1203875
Total TAM width = 32							
15, 17	831177	15, 17	551778	5, 27	1405440	9, 23	895443
10, 7, 15	933906	1, 15, 16	551778	1, 7, 24	1009262	4, 5, 23	896732
1, 1, 7, 23	834826	1, 5, 11, 15	584524	1, 1, 4, 26	3695454	2, 2, 5, 23	909796
Total TAM width = 40							
9, 31	831177	13, 27	544579	5, 35	1241245	16, 24	754291
7, 16, 17	859998	1, 12, 27	544579	1, 16, 23	999931	8, 9, 23	715444
1, 10, 1, 28	790995	1, 1, 12, 26	544579	1, 10, 1, 28	2699795	8, 9, 11, 12	733392
Total TAM width = 48							
16, 32	808844	17, 31	544579	5, 43	1405440	24, 24	625827
7, 13, 28	843401	1, 12, 35	544579	7, 16, 25	995650	10, 15, 23	599357
1, 13, 13, 21	754307	1, 1, 12, 34	544579	4, 13, 1, 30	1903700	4, 6, 15, 23	599357
Total TAM width = 56							
16, 40	808844	16, 40	544579	5, 51	1152364	10, 46	524202
10, 13, 33	832498	1, 12, 43	544579	13, 19, 24	995522	3, 7, 46	524202
7, 13, 10, 26	661844	1, 1, 12, 42	544579	4, 1, 25, 26	1121150	1, 9, 23, 23	519712
Total TAM width = 64							
23, 41	808844	16, 48	544579	5, 59	1008220	18, 46	471094
10, 13, 41	689711	1, 12, 51	544579	4, 25, 35	995522	8, 10, 46	460930
7, 13, 10, 34	661844	1, 1, 12, 50	544579	1, 10, 25, 28	995522	3, 5, 8, 47	459781
d695				p22810			
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
5, 11	44815	7, 9	44261	6, 10	324902	6, 10	468975
2, 5, 9	44809	2, 5, 9	42424	4, 4, 8	321172	4, 6, 6	447359
1, 1, 5, 9	44809	1, 2, 4, 9	42594	1, 1, 1, 13	309905	3, 3, 5, 5	439217
Total TAM width = 24							
3, 21	31170	5, 19	29345	11, 13	308674	8, 16	339555
1, 2, 21	31132	1, 4, 19	28955	1, 10, 13	282849	6, 7, 11	314219
1, 1, 1, 21	31132	1, 1, 4, 18	28955	1, 4, 4, 15	271146	5, 5, 6, 8	299220
Total TAM width = 32							
16, 16	26578	12, 20	24728	15, 17	270379	1, 31	236828
4, 9, 19	21856	5, 9, 18	21527	6, 10, 16	258978	4, 12, 16	246461
1, 2, 10, 19	22427	1, 4, 9, 18	21518	1, 4, 13, 14	240120	5, 7, 10, 10	240685
Total TAM width = 40							
20, 20	22848	1, 39	18837	19, 21	265452	15, 25	231137
5, 9, 26	21005	6, 17, 17	18448	10, 13, 17	230462	4, 16, 20	205187
1, 4, 9, 26	21005	3, 6, 13, 18	17677	3, 7, 13, 17	220194	1, 1, 1, 37	187268
Total TAM width = 48							
16, 32	20314	1, 47	14974	19, 29	265452	17, 31	185430
4, 12, 32	18799	1, 9, 38	15084	10, 13, 25	226844	1, 16, 31	172499
2, 2, 12, 32	18799	1, 1, 1, 45	14974	4, 10, 13, 21	214379	1, 1, 16, 30	179501
Total TAM width = 56							
24, 32	19457	10, 46	14974	19, 37	245234	20, 36	175946
5, 17, 34	13575	1, 1, 54	14974	10, 13, 33	226813	1, 18, 37	160122
1, 4, 17, 34	13575	6, 16, 16, 18	12834	4, 10, 13, 29	189146	1, 15, 19, 21	145417
Total TAM width = 64							
32, 32	19457	12, 52	13732	19, 45	245234	17, 47	173325
7, 17, 40	19457	7, 20, 37	12547	10, 13, 41	223079	1, 22, 41	130785
3, 6, 16, 39	12841	8, 17, 17, 22	11420	1, 10, 13, 40	181698	1, 1, 21, 41	133256

Table 12Test time under $\alpha=0.80$, Maximum priority on test time.

$\alpha=0.80$							
p34392				p93791			
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
8, 8	1081570	1, 15	1007794	3, 13	2103700	7, 9	1800390
1, 7, 8	1005088	5, 5, 6	1035012	1, 1, 14	2229152	3, 5, 8	1790275
1, 1, 1, 13	1391607	1, 5, 5, 5	1023511	1, 1, 2, 12	4023112	2, 4, 5, 5	1800980
Total TAM width = 24							
9, 15	1064994	12, 12	821123	4, 20	1406320	1, 23	1230138
1, 10, 13	1049435	7, 8, 9	720854	1, 4, 19	1412113	7, 8, 9	1192496
1, 1, 1, 21	1391607	4, 4, 8, 8	675774	1, 1, 1, 21	3823747	5, 5, 6, 8	1203875
Total TAM width = 32							
15, 17	831177	16, 16	544579	5, 27	1405440	9, 23	895443
10, 7, 15	933906	1, 15, 16	551778	1, 7, 24	1009262	4, 5, 23	896807
1, 1, 7, 23	834826	1, 1, 15, 15	576500	1, 1, 4, 26	3695454	4, 6, 6, 16	910026
Total TAM width = 40							
9, 31	831177	13, 27	544579	5, 35	1241245	16, 24	754291
7, 16, 17	859998	1, 12, 27	544579	1, 16, 23	999931	8, 9, 23	715444
1, 10, 1, 28	790995	1, 1, 12, 26	544579	1, 10, 1, 28	2699795	2, 3, 12, 23	732635
Total TAM width = 48							
16, 32	808844	17, 31	544579	5, 43	1405440	24, 24	625827
7, 13, 28	843401	1, 12, 35	544579	7, 16, 25	995650	10, 15, 23	599357
1, 13, 13, 21	754307	1, 1, 12, 34	544579	4, 13, 1, 30	1903700	4, 6, 15, 23	599357
Total TAM width = 56							
16, 40	808844	16, 40	544579	5, 51	1152364	10, 46	524202
10, 13, 33	832498	1, 12, 43	544579	13, 19, 24	995522	3, 7, 46	524202
7, 13, 10, 26	661844	1, 1, 12, 42	544579	4, 1, 25, 26	1121150	1, 9, 23, 23	519712
Total TAM width = 64							
23, 41	808844	16, 48	544579	5, 59	1008220	18, 46	471094
10, 13, 41	689711	1, 12, 51	544579	4, 25, 35	995522	8, 10, 46	460930
7, 13, 10, 34	661844	1, 1, 12, 50	544579	1, 10, 25, 28	995522	8, 10, 23, 23	459394
d695				p22810			
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
5, 11	44815	7, 9	44261	6, 10	324902	6, 10	468975
2, 5, 9	44809	2, 5, 9	42424	4, 4, 8	321172	4, 6, 6	447359
1, 1, 5, 9	44809	1, 3, 3, 9	42644	1, 1, 1, 13	309905	3, 3, 5, 5	438437
Total TAM width = 24							
3, 21	31170	5, 19	29345	11, 13	308674	8, 16	339555
1, 2, 21	31132	2, 4, 18	28546	1, 10, 13	282849	6, 7, 11	314219
1, 1, 1, 21	31132	1, 1, 4, 18	28955	1, 4, 4, 15	271146	5, 5, 6, 8	299220
Total TAM width = 32							
16, 16	26578	13, 19	23056	15, 17	270379	1, 31	236828
4, 9, 19	21856	5, 9, 18	21527	6, 10, 16	258978	4, 12, 16	246332
1, 2, 10, 19	22427	1, 4, 9, 18	21518	1, 4, 13, 14	240120	5, 7, 10, 10	240481
Total TAM width = 40							
20, 20	22848	1, 39	18837	19, 21	265452	15, 25	231137
5, 9, 26	21005	6, 17, 17	18448	10, 13, 17	230462	4, 16, 20	205187
1, 4, 9, 26	21005	1, 4, 17, 18	17677	3, 7, 13, 17	220194	1, 1, 1, 37	191797
Total TAM width = 48							
16, 32	20314	1, 47	14974	19, 29	265452	17, 31	185430
4, 12, 32	18799	1, 9, 38	15084	10, 13, 25	226844	1, 16, 31	172499
2, 2, 12, 32	18799	1, 1, 1, 45	14974	4, 10, 13, 21	214379	1, 1, 16, 30	175225
Total TAM width = 56							
24, 32	19457	10, 46	14974	19, 37	245234	20, 36	175946
5, 17, 34	13575	17, 17, 22	14644	10, 13, 33	226813	1, 18, 37	160122
1, 4, 17, 34	13575	6, 16, 16, 18	12834	4, 10, 13, 29	189146	7, 13, 16, 20	145417
Total TAM width = 64							
32, 32	19457	12, 52	13732	19, 45	245234	17, 47	173325
7, 17, 40	19457	3, 24, 37	12547	10, 13, 41	223079	1, 22, 41	130785
3, 6, 16, 39	12841	8, 17, 17, 22	11420	1, 10, 13, 40	181698	1, 1, 21, 41	133256

Table 13Test time under $\alpha=1.00$, Full priority on test time.

$\alpha=1.00$							
p34392				p93791			
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
7, 9	1 134 919	1, 15	1 007 794	7, 9	1 800 413	7, 9	1 800 390
1, 7, 8	999 543	1, 5, 10	1 049 089	4, 4, 8	1 779 401	3, 5, 8	1 790 275
1, 4, 4, 7	1 288 341	1, 5, 5, 5	1 023 511	1, 4, 4, 7	2 179 527	2, 3, 3, 8	1 809 815
Total TAM width = 24							
9, 15	913 550	12, 12	821 123	7, 17	1 259 711	1, 23	1 230 138
7, 7, 10	762 841	6, 9, 9	738 059	4, 4, 16	1 197 683	7, 8, 9	1 205 904
1, 1, 1, 21	798 449	2, 5, 8, 9	698 637	1, 7, 7, 9	1 337 682	1, 5, 6, 12	1 212 417
Total TAM width = 32							
11, 21	843 301	16, 16	544 579	9, 23	894 463	9, 23	895 443
10, 7, 15	665 445	6, 10, 16	544 579	4, 4, 24	910 957	4, 5, 23	897 595
4, 7, 10, 11	684 524	1, 1, 15, 15	576 500	1, 10, 10, 11	944 807	7, 8, 8, 9	898 721
Total TAM width = 40							
13, 27	752 782	13, 27	544 579	15, 25	778 296	16, 24	754 291
7, 13, 20	552 231	1, 16, 23	544 579	7, 10, 23	817 805	8, 9, 23	714 942
1, 4, 16, 19	584 301	1, 1, 16, 22	544 579	1, 4, 16, 19	890 145	5, 5, 7, 23	717 557
Total TAM width = 48							
13, 35	693 465	13, 35	544 579	15, 33	758 806	24, 24	625 827
7, 16, 25	546 152	1, 16, 31	544 579	7, 16, 25	722 042	10, 15, 23	599 357
1, 7, 19, 21	544 579	1, 1, 16, 30	544 579	4, 13, 13, 18	713 347	7, 9, 9, 23	602 782
Total TAM width = 56							
13, 43	637 606	13, 43	544 579	11, 45	662 686	10, 46	524 202
7, 16, 33	540 069	1, 16, 39	544 579	13, 16, 27	635 095	5, 5, 46	524 305
1, 7, 19, 29	544 579	1, 1, 16, 38	544 579	1, 7, 22, 26	664 447	2, 8, 23, 23	519 247
Total TAM width = 64							
13, 51	559 758	16, 48	544 579	15, 49	630 365	18, 46	471 094
7, 16, 41	534 212	1, 16, 47	544 579	16, 22, 26	572 342	8, 10, 46	460 930
7, 10, 19, 28	544 330	1, 1, 16, 46	544 579	1, 13, 22, 28	604 553	3, 3, 12, 46	454 913
d695							
p22810							
Total TAM width = 16							
Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time	Wu et al. [10] TAM partition	[10] test time	Our TAM partition	Our test time
7, 9	24 257	7, 9	44 261	5, 11	289 986	6, 10	468 975
4, 5, 7	22 984	2, 5, 9	42 424	4, 4, 8	275 537	4, 5, 7	449 776
1, 1, 4, 10	27 403	3, 3, 5, 5	43 193	1, 4, 4, 7	286 734	2, 4, 5, 5	438 437
Total TAM width = 24							
5, 19	21 953	5, 19	29 345	11, 13	263 892	8, 16	339 555
1, 4, 19	20 764	2, 4, 18	28 546	1, 10, 13	254 732	5, 8, 11	312 070
1, 3, 4, 16	16 549	1, 1, 5, 17	28 292	1, 1, 10, 12	249 274	5, 5, 6, 8	299 220
Total TAM width = 32							
11, 21	20 934	13, 19	23 056	7, 25	230 925	1, 31	236 828
4, 10, 18	16 085	5, 9, 18	21 527	4, 10, 18	232 810	6, 10, 16	243 814
2, 4, 10, 16	13 505	1, 2, 11, 18	21 512	1, 10, 10, 11	205 831	5, 7, 10, 10	234 452
Total TAM width = 40							
19, 21	18 604	1, 39	18 837	13, 27	215 506	15, 25	231 137
4, 17, 19	13 095	4, 18, 18	17 677	10, 13, 17	201 221	1, 1, 38	200 254
4, 4, 13, 19	11 282	3, 6, 13, 18	17 677	4, 7, 10, 19	181 687	4, 7, 11, 18	196 069
Total TAM width = 48							
21, 27	17 331	1, 47	14 974	13, 35	205 782	17, 31	185 430
7, 19, 22	11 808	1, 9, 38	15 084	7, 13, 28	172 191	11, 17, 20	169 635
3, 4, 7, 34	9 522	5, 11, 14, 18	14 921	7, 10, 10, 21	169 392	8, 8, 11, 21	172 601
Total TAM width = 56							
23, 33	15 392	10, 46	14 974	15, 41	204 155	20, 36	175 946
4, 19, 33	11 808	7, 17, 32	13 875	10, 13, 33	153 282	14, 21, 21	145 867
2, 4, 16, 34	8 452	5, 16, 16, 19	12 941	4, 10, 16, 26	162 818	1, 17, 17, 21	145 417
Total TAM width = 64							
23, 41	13 984	12, 52	13 732	15, 49	179 688	17, 47	173 325
5, 22, 37	9 731	7, 17, 40	12 846	13, 19, 32	142 210	1, 24, 39	139 939
13, 16, 16, 19	7 923	5, 8, 19, 32	10 985	10, 13, 19, 22	157 460	1, 2, 22, 39	129 624

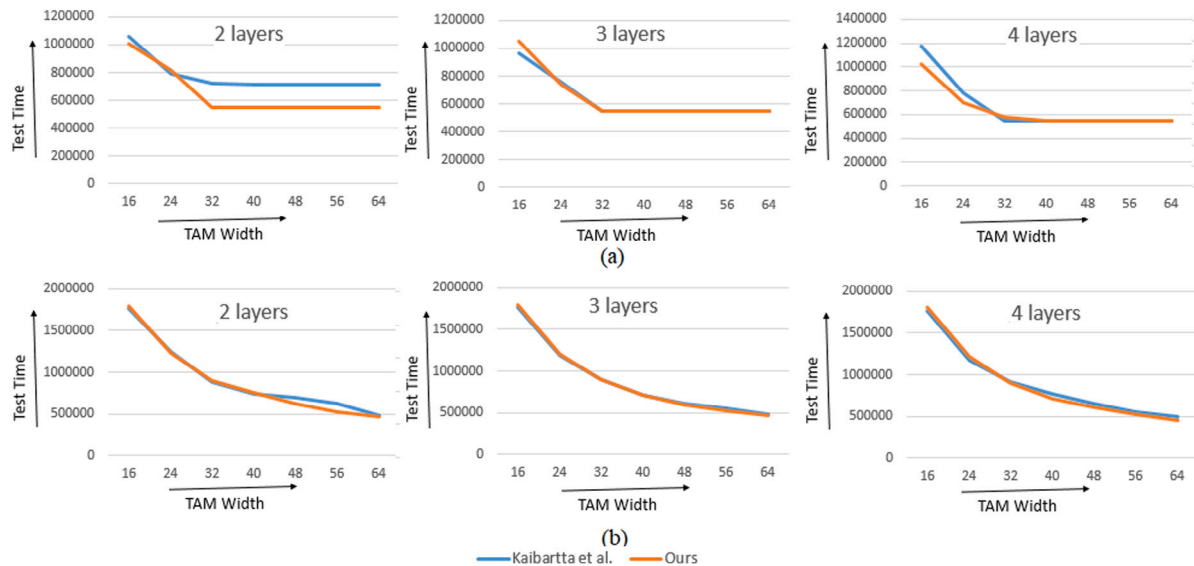


Fig. 9. Summarized result for Table 14 (a) result for p34392 (b) result for p93791.

Table 14
Comparison with [24].

No. of TAM partitions	TAM width	p34392		p93791	
		[24] test time	Ours test time	[24] test time	Ours test time
2	16	1 063 171	1 007 794	1 769 435	1 800 390
	24	792 439	821 123	1 242 931	1 230 138
	32	721 571	544 579	883 629	895 443
	40	714 855	544 579	744 093	754 291
	48	714 855	544 579	699 371	625 827
	56	714 855	544 579	628 289	524 202
	64	714 855	544 579	472 415	471 094
3	16	964 782	1 049 089	1 761 151	1 790 275
	24	759 427	738 059	1 192 801	1 205 904
	32	544 579	544 579	904 688	897 595
	40	544 579	544 579	711 570	714 942
	48	544 579	544 579	616 006	599 357
	56	544 579	544 579	550 229	524 305
	64	544 579	544 579	476 606	460 390
4	16	1 174 703	1 023 511	1 772 245	1 809 815
	24	781 178	698 637	1 177 812	1 212 417
	32	544 579	576 500	916 816	898 721
	40	544 579	544 579	763 539	717 557
	48	544 579	544 579	646 930	602 782
	56	544 579	544 579	548 536	519 247
	64	544 579	544 579	496 578	454 913

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] S. Rahimian, V.F. Pavlidis, G.D. Micheli, Scheduling tests for VLSI systems under power constraints, inter-plane communication methods for 3-d ics, *J. Low Power Electron.* 8 (2012).
- [2] ITC 2002 SoC benchmarking initiative.
- [3] R. Weerasekera, L. Zheng, D. Pamunuwa, H. Tenhunen, Extending systems-on-chip to the third dimension: Performance, cost and technological tradeoffs, in: *Proc. Int. Conference on Computer-Aided Design*, 2007, pp. 212–219.
- [4] V. Iyengar, K. Chakrabarty, E.J. Marinissen, Test wrapper and test access mechanism co-optimization for system-on-chip, *J. Electron. Test. Theory Appl.* 18 (2002) 213–230.
- [5] Y. Zorian, E.J. Marinissen, M. Lousberg, S.K. Goel, Wrapper design for embedded core test, in: *Proceedings International Test Conference 2000* (IEEE Cat. No. 00CH37159), 2000, pp. 911–920.
- [6] Y. Zorian, E.J. Marinissen, S. Dey, Testing embedded core-based system chips, *Computer* 32 (6) (1999) 52600018–52609162.
- [7] P. Venkataramani, V.D. Agrawal, Reducing test time of power constrained test by optimal selection of supply voltage, in: *Proc. 26th International Conf. VLSI Design*, 2013, pp. 273–278.
- [8] K. Chakrabarty, Test scheduling for core-based systems using mixed-integer linear programming, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 19 (10) (2000) 1163–1174.
- [9] V. Iyengar, K. Chakrabarty, E.J. Marinissen, Test wrapper and test access mechanism co-optimization for system-on-chip, *J. Electron. Test.: Theory Appl.* 18 (2002) 211–228.
- [10] X. Wu, Y. Chen, K. Chakrabarty, Y. Xie, Test-access mechanism optimization for core-based three-dimensional socs, *Microelectron. J.* 41 (2010) 601–615.
- [11] M. Pradhan, C. Giri, H. Rahaman, D.K. Das, Optimizing test time for core-based 3-D integrated circuits by a technique of Bi-partitioning, in: *Proc. Design and Test Symposium (EWDTS)*, 2014 East-West, 2014, pp. 1–4.
- [12] V. Iyengar, K. Chakrabarty, E.J. Marinissen, On using rectangle packing for SoC wrapper/TAM co-optimization, in: *Proc. 20th IEEE VLSI Test Symp.*, 2002, pp. 253–258.
- [13] S. Banerjee, S. Majumder, B.B. Bhattacharya, Power-aware test optimization for core-based 3D-SOCs under TSV-constraints, in: *Proc. 2016 IFIP/IEEE International Conference on Very Large Scale Integration, VLSI-SoC 2016*, Tallinn, Estonia, September 26–28, 2016, 2016, pp. 1–6.
- [14] P. Girard, N. Nicolici, X. Wen, *Power-Aware Testing and Test Strategies for Low Power Devices*, Springer, 2010.
- [15] S. Koranne, Formulating soc test scheduling as a network transportation problem, *IEEE Trans. CAD Integr. Circuits Syst.* 21 (12) (2002) 1517–1525.
- [16] E. Larsson, H. Fujiwara, System-on-chip test scheduling with reconfigurable core wrappers, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 14 (3) (2006) 305–309.
- [17] E. Larsson, J. Pouget, Z. Peng, Defect-aware SOC test scheduling, in: *Proc. 22nd IEEE VLSI Test Symp. (VTS)*, 2004, pp. 359–364.
- [18] Q. Zhou, K.J. Balakrishnan, Test cost reduction for soc using a combined approach to test data compression and test scheduling, in: *2007 Design, Automation and Test in Europe Conference and Exposition, DATE 2007*, Nice, France, April 16–20, 2007, 2007, pp. 39–44.
- [19] B. SenGupta, D. Nikolov, U. Ingelsson, E. Larsson, Test planning for core-based integrated circuits under power constraints, *J. Electron. Test.* 33 (1) (2017) 7–23.
- [20] X. Cui, W. Cheng, C.L. Lee, Effective soc test scheduling under peak power constraints based on the ant colony algorithm, *Microelectron. Comput.* (2011).
- [21] B. Noia, S. Goel, K. Chakrabarty, E.J. Marinissen, J. Verbree, Test-architecture optimization for TSV-based 3Dstacked ICs, in: *Proc. Test Symposium (ETS)*, 2010 15th IEEE European, 2010, pp. 24–29.
- [22] S. Banerjee, S. Majumder, B.B. Bhattacharya, Test-time reduction for power-aware 3D-SoC, in: *Proc. 31st International Conference on VLSI Design 17th International Conference on Embedded Systems, VLSID 2018*, Pune, India, January 8–10, 2018, 2018, pp. 1–6.
- [23] 3D SoC Test Benchmarks <http://3dsocbench.ece.wisc.edu/>.
- [24] T. Kaibartta, C. Giri, H. Rahaman, D.K. Das, Approach of genetic algorithm for power-aware testing of 3D IC, *IET Comput. Digit. Tech.* 13 (5) (2019) 383–396.